

MISC: Flexible Data Management for Private Smart Contract

Yuxian Li , Xiaoguo Li , Guomin Yang , Senior Member, IEEE, Haoxin Tu , Member, IEEE, Jianfei Sun , and Robert H. Deng , Fellow, IEEE

Abstract—Private Smart Contracts (PSCs) aim to settle the privacy protection issue for sensitive data on the blockchain (a.k.a., on-chain data) while inheriting the computation functionality of general smart contracts. However, most of the existing PSCs focus on secure computation and ignore the secure and flexible management of on-chain data, making it difficult to support flexible user permission modes efficiently. Further, although a few on-chain data management schemes have been proposed to manage data on the blockchain, they provide insufficient support for scenarios where data must be managed concurrently by managers with different privilege levels. This paper proposes MISC, a novel solution that features efficient and flexible data management capabilities for managing on-chain data in private smart contracts. MISC leverages a novel Double Trapdoor Homomorphic Proxy Re-encryption (DT-HPRE) scheme and Trusted Execution Environments (TEEs) to ensure (1) low-overhead, flexible permission control over on-chain data and (2) a harmonious mechanism for both privileged and constrained managers. We prove the security of MISC under the Universal Composability (UC) framework to demonstrate its data confidentiality guarantees, user revocation, and computation integrity. Further, we implement MISC, and the performance evaluation shows that on-chain data access for one hundred data points is practically achieved in 5.8 s (under privileged managers) and 9.7 s (under constrained managers) using its off-chain design.

Index Terms—Private smart contract, trusted execution environment, data management.

I. INTRODUCTION

PRIVATE Smart Contracts (PSCs) enable data owners to store data on-chain and allow data users to execute computations on the data without revealing the underlying data content. By incorporating Non-Interactive Zero-Knowledge (NIZK)

proofs [1] and Trusted Execution Environments (TEEs) [2], PSCs ensure that on-chain data can be updated according to the smart contract's logic while preserving confidentiality. Due to their privacy-preserving design and programmability, PSCs have been widely applied in diverse privacy-demanding domains, such as financial transactions and healthcare data management [3], [4], [5], [6]. However, existing PSCs, such as Zkay [7], Zeestar [8], and Hawk [9], only support all-or-nothing access permissions, in which all data must be fully shared or entirely hidden, lacking differentiated access control across users and concurrent management by data managers with different privilege levels [10], [11]. This simplistic design not only limits data circulation but also imposes impractical access control constraints in collaborative ecosystems, where diverse stakeholders need differentiated access to specific data.

To address these limitations, we identify the following key requirements for practical data management in PSCs: (R1) supporting various user permission modes (e.g., defining accessible data fields and permitted operations)—for example, physicians can be granted permissions to read only specific subsets of patient medication records, while researchers might access solely aggregated statistical summaries; (R2) dynamically adjusting user permissions (i.e., revoking or changing granted permissions); and (R3) enabling data management by managers with varying privilege levels determined by data owners. A *privileged manager*, e.g., the government trusted by its citizens, can grant *comprehensive computational permissions*, making it capable of granting a data user, like a tax auditor, to assess annual tax liabilities over an owner's financial data. Conversely, a *constrained manager* is limited to managing owner-approved access rights, ensuring all permissions remain governed by data owners.

A. Limitations

Unfortunately, achieving these requirements through native PSC implementations is impractical. Typically in the implementation of PSCs, the developers has to *hardcode* user permissions directly into the contract's source code [10], [12], tightly coupling permission enforcement with contract logic. This design imposes severe practical constraints. That is, creating new permission modes (R1) or updating existing ones (R2) requires modifying and redeploying the entire contract, incurring prohibitive overhead for even minor adjustments. Furthermore, since PSCs usually restrict managers to ciphertext operations without access to plaintext, they cannot support managers with different privilege levels (R3). This lack of flexibility makes PSC-native data management impractical for real-world use. Hence,

Received 22 October 2025; revised 12 March 2026; accepted 2 June 2026. Date of publication 8 June 2026; date of current version 1 September 2026. This work was supported in part by the National Research Foundation, Singapore and Infocomm Media Development Authority under its Trust Tech Funding Initiative, in part by Lee Kong Chian Chair Professor fund, in part by Singapore Management University, in part by the National Natural Science Foundation of China under Grant 62502056, and in part by Chongqing Young Talents Program under Grant CSTB2025YCJH-KYXM0068. (Corresponding author: Jianfei Sun.)

Yuxian Li, Guomin Yang, Haoxin Tu, and Robert H. Deng are with the School of Computing and Information Systems, Singapore Management University, Singapore 188065 (e-mail: yuxianli@smu.edu.sg; gmyang@smu.edu.sg; haoxintu@smu.edu.sg; robertdeng@smu.edu.sg).

Xiaoguo Li is with the College of Computer Science, Chongqing University, Chongqing 400044, China (e-mail: csxgli@cqu.edu.cn).

Jianfei Sun is with the University of Electronic and Science Technology of China, Chengdu 610054, China (e-mail: jianfei.sun@uestc.edu.cn).

This article has supplementary downloadable material available at <https://doi.org/10.1109/TDSC.2026.3701041>, provided by the authors.

Digital Object Identifier 10.1109/TDSC.2026.3701041

decoupling user permission enforcement from PSCs emerges as a practical solution to achieve flexible data management [11], [13]. This paradigm enables data owners to store encrypted data on-chain while delegating permission enforcement to off-chain managers. While these solutions [11], [13] address data management for PSCs, they encounter two notable limitations:

L1. High overhead to support flexible user permission mode: One approach, like Oasis Labs [13], to enable flexible access to on-chain data is to replicate blockchain data to a traditional database and leverage conventional data management techniques. Nevertheless, such a replication-based strategy introduces the following issues: 1) synchronization problems between blockchains and off-chain databases must be resolved to maintain consistency, and 2) additional storage is required for maintaining the replicated data. Furthermore, Oasis Labs [13] relies on a centralized SQL database, which will result in a single point of failure and raise security concerns. Despite enabling off-chain access policy updates without on-chain costs, this approach compromises the decentralization inherent to blockchain systems, motivating the need for more efficient and scalable solutions for managing on-chain data.

L2. Inadequate support for managers with diverse privilege levels: When all managers have equal authority to authorize computational operations over sensitive on-chain data, privacy risks arise in practice. For instance, in the context of healthcare data, both a hospital administrator (as a privileged manager) and a third-party analytics provider (as a constrained manager) could authorize a researcher with full computational permissions over complete patient medical records. The administrator should be able to authorize comprehensive computations on complete patients' medical histories within TEEs, while the analytics provider should only be allowed to authorize re-encryption of patient-approved data subsets for specific researchers, ensuring that only explicitly consented information is transformed and shared with third parties. However, existing PSC-based data management approaches assume all managers operate with the same privilege scope. CALYPSO [11], for instance, designates a committee as a constrained manager to handle re-encryption of on-chain data for data users. Similarly, Oasis Labs [13] assumes the presence of privileged managers, who operate centralized databases and enforce conventional access control policies. In contrast to these uniform privilege models, real-world applications often demand hierarchical management structures where managers possess different privilege levels over the same dataset. Thus, supporting managers with different privilege levels during on-chain data management remains a significant challenge.

B. Our Contributions

To address these limitations, we propose MISC, a flexible data management system, leveraging TEEs [2], [14] to separate permission enforcement from smart contract logic. This separation enables a key advantage: unlike existing TEE-backed PSC systems such as Ekiden [15] and Hawk [9], which need to redesign blockchain frameworks and thus become incompatible with Ethereum, MISC works as a parallel plugin. It integrates with existing PSCs on different blockchains without altering their execution semantics. The key ideas behind our design are as follows.

First, to achieve a flexible permission mode with minimal overhead, we must eliminate the need for additional centralized databases while maintaining efficiency and confidentiality. Our basic approach embeds the data owner's decryption capability within TEEs, where managers maintain user permissions through the TEE. A naive solution would have PSC users generate an additional ciphertext with a new public key alongside the PSC-generated ciphertext when updating on-chain data. Then, the corresponding decryption trapdoor will be sent to the TEE to achieve user permission management. This approach forces the TEE to store numerous decryption keys and maintain complex ciphertext-key mapping tables. However, TEE hardware constraints—such as Intel SGX's 128 MB EPC [2]—make storing numerous keys and complex mapping tables impractical.

To address this challenge, we redesign the underlying encryption scheme in PSCs to support a double-trapdoor property [16], in which a single ciphertext can be decrypted using either of two independent trapdoors. With this property, in MISC, one trapdoor is held by the data owner while the other serves as a shared trapdoor embedded in the TEE. This design enables the TEE to store only a single trapdoor for all on-chain data decryption instead of maintaining a large number of decryption keys and complex ciphertext-key mapping tables.

Second, to address the limitation of single privilege models in existing PSC-based data management, we must enable managers with diverse privilege levels to have differentiated authorization capabilities on the same dataset. Specifically, privileged managers should be able to authorize comprehensive computations on complete data within TEEs, while constrained managers should only be able to authorize users to access the owner-approved re-encrypted data subsets. A naive solution is to employ two separate encryption mechanisms: one standard encryption scheme for privileged managers to decrypt-then-compute data in TEEs, and another proxy re-encryption scheme for constrained managers' data transformation in TEE. However, this approach requires storing multiple ciphertext versions for the same data—one for each privilege level—leading to additional storage overhead.

To address this challenge, MISC further enhances the underlying encryption scheme to support double-trapdoor decryption and proxy re-encryption simultaneously. The first trapdoor, held by data owners (e.g., citizens), enables direct decryption and the creation of re-encryption keys sent to the TEE. The second trapdoor, embedded in the TEE, supports internal decryption for computation. This design avoids the need for multiple ciphertext versions, allowing a single on-chain ciphertext to be securely managed by multiple managers with varying privilege levels. *Constrained managers* (e.g., a visa application) can authorize users (e.g., visa officers) to obtain data re-encrypted by the TEE using proxy keys. *Privileged managers* (e.g., government tax agencies) can permit users (e.g., tax auditors) to leverage the second trapdoor within TEEs to decrypt on-chain data within TEEs, execute computations (e.g., calculating annual tax liabilities), and return only the computational result. This grants auditors access to the computed tax amount without exposing raw financial records. The key advantage of this unified scheme is that multiple managers with varying privilege levels can securely manage the same on-chain data simultaneously, eliminating the need for multiple ciphertext versions and significantly reducing storage overhead.

Contributions: Our contributions are as follows:

- *A novel double-trapdoor homomorphic proxy re-encryption (DT-HPRE) scheme:* We propose DT-HPRE, a novel cryptographic primitive that unifies double-trapdoor mechanisms with homomorphic proxy re-encryption. Unlike existing schemes that often trade off functionality for security, our construction formally supports three critical properties simultaneously: 1) dual-decryption capabilities for hierarchical access control, 2) proxy re-encryption for secure data sharing, and 3) additive homomorphism. This unified design eliminates the need for multiple ciphertext versions across different privilege-level managers while ensuring seamless integration with diverse PSCs.
- *Formal Security Analysis under the UC Framework:* We provide a rigorous security model for our system. We formally prove that MISC achieves data confidentiality, integrity of computation, and secure user revocation within the Universal Composability (UC) framework [17].
- *A flexible TEE-assisted data management scheme:* We introduce MISC, a flexible on-chain data management scheme. By leveraging DT-HPRE and TEEs, MISC provides a management layer that: (a) preserves the original PSC architecture and execution model, (b) maintains its privacy guarantees and computational capabilities, and (c) adheres to diverse cryptographic assumptions across different PSC frameworks. This design ensures that MISC integrates with most existing PSCs—including both HE-based and non-HE-based systems—as a drop-in enhancement rather than requiring infrastructure modifications. Consequently, MISC resolves the “compatibility bottleneck” that limits the extensibility of prior systems like Zeestar [8].
- *Implementation and Efficiency Evaluation:* We implement DT-HPRE and MISC schemes to demonstrate their real-world feasibility. Our comprehensive evaluation shows that by shifting heavy management logic off-chain via TEEs, the system achieves operation completion in seconds. We validate the scalability of our solution by measuring performance across various on-chain dataset sizes and testing latency under various numbers of concurrent users, confirming its practicality.

C. Applications

To illustrate the potential applications of MISC, we present two use cases to demonstrate the versatility and effectiveness of our approach in handling on-chain data.

Supply Chain: A representative application of PSCs is supply-chain logistics, where shipment records (e.g., invoices, provenance, IoT sensor logs, and delivery proofs) are confidentially stored on-chain to prevent business leakage [18]. However, *all-or-nothing* sharing in traditional PSC frameworks cannot meet the need for textit differentiated access to the same dataset by different stakeholders (e.g., insurers, customs, and regulators). At the same time, such permissions often need to be updated dynamically. MISC addresses these needs by decoupling permission enforcement from PSC logic and enabling flexible data management. For example, the logistics platform operator (i.e., *constrained manager*) can authorize an insurance agency to access only owner-approved subsets (R1), such as a shipment identifier and proof of delivery, by instructing the TEE to output re-encrypted ciphertexts for these fields. Meanwhile, a government (i.e., a *privileged manager*) can authorize *computation-only* auditing over complete records to a specific

auditor (R3), returning only the audit outcome (e.g., whether the cold-chain compliance rule was violated) without disclosing raw business data.

Financial Audit: Our data management solution also enhances the efficiency and security of financial audits, specifically for on-chain digital assets issued via PSCs. The privacy-preserving feature of PSCs makes the on-chain trading data private for any parties, but it also hinders official regulators from auditing digital asset transactions. With MISC, auditors can obtain required audit results under a least-disclosure principle. A privileged manager (e.g., a regulatory authority) can authorize a specific auditor to collect the required result (e.g., total trading volume within a period) by accessing the TEE to decrypt and compute aggregate audit metrics over the full encrypted ledger(R1/R3), while keeping individual trades private. In contrast, constrained managers (e.g., compliance service providers) can be restricted to re-encrypting only owner-approved views (e.g., per-asset summaries) for specific auditors, preventing excessive disclosure (R3). Moreover, audit permissions can be dynamically updated or revoked (R2) to meet evolving regulatory requirements and audit scopes.

II. PRELIMINARIES

A. Private Smart Contract

A Private Smart Contract (PSC) is a self-executing contract with the terms of the agreement directly written into code, where sensitive data is encrypted and stored on the blockchain [12]. We take Zeestar as an example [8] to introduce PSCs. Zeestar introduces a framework, comprising a language and a compiler, which empowers developers, even those without deep cryptographic expertise, to create and deploy such contracts. It achieves privacy by enabling developers to specify data ownership and privacy constraints through annotations; the compiler then automatically enforces these by synergistically combining Non-Interactive Zero-Knowledge (NIZK) proofs with additively homomorphic encryption. This approach notably allows for secure operations on “foreign data”—data owned by parties other than the transaction caller—a capability crucial for applications like private payment systems, thereby significantly expanding the expressiveness and utility of private smart contracts on public blockchains like Ethereum.

B. Cryptographic Building Blocks

Non-Interactive Zero-Knowledge (NIZK) Scheme: Let us consider the relation $R(x, w)$ belonging to the language $L := \{x | \exists w \text{ such that } (x, w) \in R\}$, where x represents a statement and w is its related witness. A NIZK scheme [19] is formulated as $\Xi_R = (\text{Prove}, \text{Vry})$. $\pi \leftarrow \text{Prove}(w, x)$ is a prover algorithm, take the witness w and statement x as the inputs to generate a proof π stating $(x, w) \in R$. $\{0, 1\} \leftarrow \text{Vry}(\pi, x)$ is a verifier algorithm, inputting π and x and determining if there exists a w such that $(x, w) \in R$. A NIZK scheme enables a prover to convince a verifier of the existence of a witness w for a statement x using a proof π , without revealing any information other than the fact that it knows w , so-called zero-knowledge.

Public Key Encryption (PKE) Scheme: A PKE scheme consists of three algorithms: (Gen, Enc, Dec). In detail, $\text{Gen}(\lambda)$ is a key generation algorithm to produce a key pair (pk, sk) , where λ is a security parameter, pk is the public key, and sk is the private key. $\text{Enc}_{pk}(m)$ is the encryption algorithm to

encrypt a message m into a ciphertext c using pk . $\text{Dec}_{sk}(c)$ is the decryption algorithm to decrypt c back to the plaintext m' using sk . Our solution uses the ElGamal encryption scheme [20], meeting the security against Indistinguishability under Chosen Plaintext Attack (IND-CPA). At a high level, this ensures that a PPT adversary $\mathcal{A}$, without the decryption key, cannot distinguish between the two encrypted chosen plaintexts, even after observing polynomially many ciphertexts.

Homomorphic Proxy Re-encryption (HPRE): A HPRE Scheme Ψ includes: (KeyGen, ReKeyGen, Enc, ReEnc, Dec). Compared with PKE schemes, there are two additional algorithms ReKeyGen and ReEnc in Ψ . $rk_{1 \rightarrow 2} \leftarrow \text{ReKeyGen}(sk_1, pk_2)$ generates the re-encryption key $rk_{1 \rightarrow 2}$ on input the secret key sk_1 and the public key pk_2 . $c_2 \leftarrow \text{ReEnc}(rk_{1 \rightarrow 2}, c_1)$ transfers the ciphertext c_1 under the encryption key pk_1 to the ciphertext c_2 under another key pk_2 . HPRE facilitates the delegation of decryption rights and supports homomorphic operation on ciphertexts, i.e., $\text{Dec}(\text{Enc}(m_1) \oplus \text{Enc}(m_2)) = \text{Dec}(\text{Enc}(m_1)) \oplus \text{Dec}(\text{Enc}(m_2))$.

Signature Scheme: A signature scheme Ω consists of: (KeyGen, Sign, Verify). $(\overline{sk}, \overline{pk}) \leftarrow \text{KeyGen}(\lambda)$ is run by users to generate their key pair. $\sigma \leftarrow \text{Sign}(\overline{sk}, m)$ is run by a signer to sign on a message m . $\{0, 1\} \leftarrow \text{Verify}(\overline{pk}, \sigma, m)$ is the verification algorithm to determine whether the signature σ is signed on m . The signature scheme satisfies Existential Unforgeability under Chosen Message Attack (EUF-CMA) [21]. It guarantees that any PPT adversary $\mathcal{A}$, without the secret key sk , cannot forge a valid signature σ for m , even after observing polynomially many signatures on arbitrarily chosen messages (but different from m).

C. Trusted Execution Environment (TEE)

TEE creates an isolated region in main memory, called an *enclave*, ensuring the confidentiality of loaded data and the integrity of code execution [2], [14]. Various TEEs are currently available, including Intel SGX [2]. Following the formal abstraction in [9], [22], we model the TEE as an ideal functionality $\mathcal{G}_{\text{att}}$, including two algorithms: 1) $(eid, \sigma) \leftarrow \mathcal{G}_{\text{att}}.\text{install}(\text{progs})$ loads programs *progs* into an enclave and outputs the enclave identity *eid* and an attestation σ , in specific σ states the integrity of the loaded data and code; 2) $(outp, \sigma) \leftarrow \mathcal{G}_{\text{att}}.\text{resume}(eid, inp)$ inputs the data *inp* to execute the program in the enclave *eid*, which outputs the result *outp* and an attestation σ . The attestation σ proves that the program is executed correctly, and *eid* is the enclave identity. We introduce an attestation verification mechanism *verifyAtt* to verify attestations. Any party checks the integrity of program execution via *verifyAtt*($\sigma, pk_{\text{att}}, outp$), where σ is signed by a hardware secret key sk_{att} . If the result is “true” (1), the integrity is guaranteed.

D. Distributed Ledger System

A distributed ledger system, typically implemented via a blockchain, such as Bitcoin [23] and Ethereum [24], records transactions across a distributed network. The data (i.e., transactions) in the blockchain is formatted as: $((tx_{id}, name, value))$. Following the definition in [23], [25], we model the blockchain as an ideal functionality $\mathcal{G}_L$, including interfaces: 1) $\mathcal{D} \leftarrow \mathcal{G}_L.\text{Read}(D)$: It collects data set $\mathcal{D}$ from the blockchain based on its request $D = \{\dots, name_j, \dots\}$, where $\mathcal{D}$ is constructed by the data $(tx_{id}, name_j, value_j)$; 2) $1/0 \leftarrow \mathcal{G}_L.\text{Post}(tx, u_i)$:

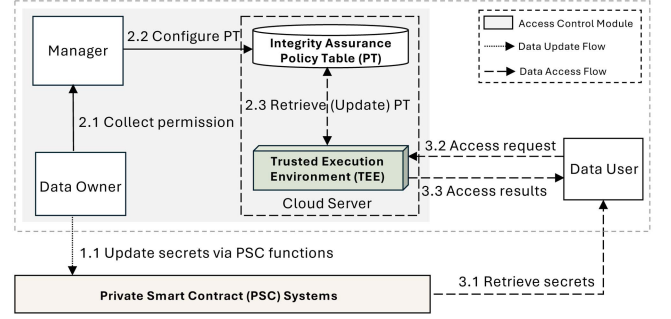


Fig. 1. Overview of MISC architecture. Access control module and data access flow are the new design parts, and data update flow is inherited from PSC systems.

This interface allows the user u_i to post the transaction tx to the ledger. Note that PSCs operate on distributed ledgers for transaction validation and integrity [12]. Meanwhile, they preserve privacy by using cryptographic techniques to restrict access to sensitive data, permitting only authorized participants to view or modify contract details.

III. OVERVIEW

A. System Model

MISC is built on the PSCs [7], [8] that serve to govern the sensitive on-chain data. The initial system setup of MISC, encompassing the deployment of these PSCs and the installation of programs within Trusted Execution Environments (TEEs), adheres to standard PSC conventions [7], [8], [15] and is typically performed by the PSC developer. This foundational PSC architecture enables MISC to operate with four key entities: a manager, data owners, a cloud server, and data users.

- **Manager:** The manager, defining the role-based access policy, determines 1) what roles should be defined for MISC, 2) what permissions (i.e., read, write, updates, or even complex computations, are allowed on these sensitive data) should be assigned to different roles, and 3) which users should be assigned a particular role.
- **Data Owners:** The data owners store and update sensitive data via PSCs. Data owners are responsible for approving permissions defined by the manager, thereby granting specific managers the ability to govern their on-chain data.
- **Cloud Server:** The cloud server equipped with a TEE stores the *integrity assurance Policy Tables (PTs)*, configured by the manager, to facilitate the management of user permissions for on-chain data.
- **Data Users:** Authorized users access on-chain data and request the server's TEE to perform permissioned computations based on that data, receiving only the results.

The overall workflow of MISC, shown in Fig. 1, includes: 1) *Dotted arrows*: data owners upload or update on-chain data using the interfaces provided by a PSC; 2) *Gray box*: the manager collects access approval from data owners and configures the PTs; 3) *Dashed arrows*: a data user submits his/her access request to the cloud server. The TEE then retrieves permissions via the PTs, performs defined functions, and returns the result of the access to the user.

B. Threat Model

We outline the threat models associated with each entity. First, we treat managers as *honest but curious*. They are curious about

sensitive on-chain data without actively subverting the system's operational protocols. Second, the adversary can corrupt data owners. Such an owner could attempt to undermine the permission system by intentionally providing invalid or misleading service keys, disrupting the legitimate data access of data users. Third, the adversary may corrupt any number of data users, who may bypass established permission controls or collude with other entities to access data beyond their legitimately granted scope. Lastly, a malicious cloud server might attempt to: 1) tamper with the stored policy tables or communication channels, and 2) provide outdated or false user permissions, thereby enabling collusion with unauthorized or revoked users to achieve illicit access to on-chain data. We assume the server's availability holds. Note that MISC aims to manage sensitive data already stored on the PSC, rather than ensuring the correctness and integrity of the PSC's data itself during the on-chain uploading process. The latter is a concern addressed by PSC schemes such as Zeestar and Zkay and is beyond the scope of our work.

TEE Security Considerations: Following the previous works [26], [27], we assume that the code executed within the TEE enclave is neither observed nor tampered with. Our reference implementation mitigates the risk of side-channel attacks by adopting constant-time and constant-memory coding practices [26] for all security-critical components of MISC. Under this threat model, we consider the enclave, its runtime state, and the TEE service and attestation keys to be fully trusted.

We acknowledge that real-world TEEs are vulnerable to various attacks, including side-channel attacks, incorrect attestation, and enclave key leakages. In parallel, there exists a substantial mitigation landscape—ranging from vendor patches to architectural defenses—that aims to close these gaps. Since engineering comprehensive defenses is orthogonal to our core contribution, we retain the completely trusted TEE abstraction and only briefly contextualize representative attack surfaces and defenses below. First, regarding side-channel leakage, a corrupted host may attempt to infer secrets via microarchitectural channels or memory-management channels. Representative attacks include controlled-channel [28] and transient-execution attacks that can expose enclave secrets under vulnerable CPU/TCB states [29]. To defend against such attacks, except for constant-time coding mentioned above, there are other defensive measures, including transactional execution [30], architectural isolation designs [31], and detection frameworks for privileged side-channel behaviors [32]. Second, for incorrect attestation, by extracting long-term quoting or attestation keys, attackers can undermine the attestation ecosystems [33]. There are some countermeasures to mitigate this problem, including freshness via nonces and short-lived sessions and integrating attestation-related messages into the TLS handshake procedure [34]. Third, regarding threats of direct enclave key compromise via fault injection or transient execution [35], which are often platform-dependent, we mitigate these risks through vendor TCB updates and operational practices such as key rotation [36]. Moreover, to reduce the impact of an enclave compromise, our design can be extended to a threshold-based multi-TEE deployment. In such a setting, sensitive secrets maintained inside the TEE can be split across multiple enclaves, so that no individual TEE alone holds sufficient information to reconstruct the full secret.

C. Design Goals

Our design aims to achieve the *security* goals (G1-G3) and other *non-security* goals (G4-G5):

- *G1. Data Confidentiality:* The content of sensitive on-chain data is protected from unauthorized disclosure, ensuring only entities with permissions can access it.
- *G2. Computation Correctness:* The policy tables configured by managers should reflect the correct execution of the policy update request. Similarly, the data access result required by data users is the outcome of correctly executing the granted computation on the on-chain data.
- *G3. User Revocation:* The manager can revoke a data user's access permission at any time, supporting dynamic and flexible control over future access to on-chain data. After revocation, the user can no longer access the on-chain data, but the outputs that were already obtained before revocation can not be invalidated.
- *G4. Scalability:* The design does not cause extra on-chain overhead (e.g., extra gas cost) to PSCs, i.e., securing the on-chain data management via off-chain procedures.
- *G5. Compatibility:* The on-chain data management scheme should be compatible with Ethereum-compatible PSCs, requiring only minimal modifications.

By achieving the design goals, MISC offers an efficient and flexible on-chain data management scheme that enhances data usability while preserving on-chain data privacy.

IV. SECURITY DEFINITION

To provide formal security guarantees, we employ the Global Universal Composability (GUC) framework proposed by Canetti [37], [38]. In the GUC model, security is defined by comparing the protocol execution in the real world against an idealized protocol execution in the ideal world, commonly referred to as the ideal functionality. A protocol is said to be GUC-secure if the real-world execution is indistinguishable from the ideal-world execution. Intuitively, this means that if the ideal protocol satisfies strong security properties, these guarantees will be inherited by the real-world protocol. In the following, we first outline our attack and communication models, then present the ideal functionality $\mathcal{F}_{\text{MISC}}$ for data management in private smart contracts, and finally demonstrate why $\mathcal{F}_{\text{MISC}}$ achieves the desired security properties.

Adversarial Model: Our framework represents the parties as Interactive Turing Machines (ITMs) that engage with the trusted functionality $\mathcal{F}$ through secure and authenticated communication channels. The adversary (so-called simulator) $\mathcal{S}$ is conceptualized as a PPT machine controlling the corrupted parties' inputs and outputs. Following conventions in [9], [15], our analysis adopts the static corruption model, wherein the adversary specifies the parties it intends to corrupt in advance.

Communication Model: Assume that interactions among parties occur over a synchronous communication network, enabling the MISC scheme to advance in distinct rounds. Following the definition in [17], [39], the universal ideal functionality clock effectively models this synchronous network. This model ensures that the receiving party subsequently collects messages dispatched by one party in round i at the start of round $i + 1$. Moreover, we assume that while parties submit inputs to functionalities within the same round, the actual transmission of these inputs occurs instantaneously.

Ideal Functionality: We illustrate the functionality $\mathcal{F}_{\text{MISC}}$ for MISC, capturing the security and privacy concepts relevant to the data management system as specified in Section VII.

Ideal functionality $\mathcal{F}_{\text{MISC}}$

The functionality $\mathcal{F}_{\text{MISC}}$ interacts with a manager M , data users $\{DU_i\}$ including data owners $\{DO_i\}$, a cloud server CS , a simulator $\mathcal{S}$, and a contract $\mathcal{C}$. Further, the functionality maintains a legitimate function family $\mathbb{F} = \{f_1, \dots, f_n\}$, where $f_i(a)$ inputs the secret set a and outputs a computation result res . Beside, the functionality maintains the set $\mathbb{W}$ to store the public key, the set $\mathcal{L}_{data}$ to store secrets, the policy tables $\mathcal{L}_P$, and the role set $\mathcal{L}_N$.

// e and $\bar{p}k$ are the encryption and signature verification keys.

Upon receiving (`''Initialize''`, sid) from party P (e.g., M/DU_i), send (`''keyGen''`, sid, P) to $\mathcal{S}$ and wait for (`''keyRec''`, $(e, \bar{p}k, P)$). Then, send $(e, \bar{p}k, P)$ to $CS, \mathcal{S}$ and await (`''key-confirm''`, sid, b) from $\mathcal{S}$. If $b = 1$: append $(e, \bar{p}k, P)$ to $\mathbb{W}$ and return $(\mathbb{W}, success)$ to P ; else, abort.

// T_{role} stores roles and T_{user} stores role-user permissions.

Upon receiving (`''SetupPolicy''`, sid, CS) from the manager M , check if M has setup before, if so, return $(b, \mathcal{L}_P)$. Else, let $\mathcal{L}_P = (T_{role}, T_{user}) = (\emptyset, \emptyset)$ and send (`''post-req''`, $sid, \mathcal{L}_P$) to $\mathcal{S}$ and CS . Then, await b from the simulator $\mathcal{S}$, assert $b = 1$, store $\mathcal{L}_N = \emptyset$, and return $(b, \mathcal{L}_P)$.

Upon receiving (`''Admin''`, $sid, P', eid, cmd, args, M$) from a manager M , check if cmd or $args$ is illegal (e.g., $args = (u, n)$ where $n \notin \mathcal{L}_N$, or $args = (n, f, D)$ where $f \notin \mathbb{F}$) then return an error and terminate. Then, if $\mathcal{L}_P \neq P'$, abort. If not, proceed as follows:

- if $cmd = \text{add-role/add-user}$, parse $args \rightarrow (n_{new}, S_{new}, f_{new})$ or (u_{new}, n_i) and retrieve S_i related to n_i . Note that for $cmd = \text{add-role}$, $args = (u, D, f)$; for $cmd = \text{add-user}$, $args = (u, n)$. Then, for each $DO_i \in S_{new}/S_i$, send (`''admin-req''`, $sid, args, cmd$) to $DO_i/\mathcal{S}$. Next, after receiving (str_0, str_1) from the simulator $\mathcal{S}$, proceed as follows:
 - if $str_0 = \text{''reKey''} \wedge cmd = \text{add-user}$, send (`''collected''`, $sid, str_1 = c_{DO_i \rightarrow DU_{new}}, \bar{p}k_{DO_i}$) to $\mathcal{S}, M$ and receive b from $\mathcal{S}$. If $b = 0$: return $(b, \perp)$; else append $(e_{DO_i \rightarrow DU_{new}}, DO_i) \rightarrow \mathbb{C}^{new}$. Next, append $args \leftarrow \text{append}(args, \Sigma/\mathbb{C}^{new})$.
 - if $str_0 = \text{''confirm''} \wedge str_1 = \text{''approve''} \wedge cmd = \text{add-role}$ holds, send (`''collected''`, $sid, S_{new}, \bar{p}k_{DO_i}$) to $\mathcal{S}, M$ and receive b from $\mathcal{S}$. If $b = 0$: return $(b, \perp)$; else, append $DO_i \rightarrow \Sigma$. Next, append $args \leftarrow \text{append}(args, \Sigma/\mathbb{C}^{new})$.

Then, if $cmd = \text{add-role}$, update $\mathcal{L}_N \leftarrow \text{append}(\mathcal{L}_N, n_{new})$ and $T_{role} \leftarrow \text{append}(T_{role}, args)$; if $cmd = \text{add-user}$, update $T_{user} \leftarrow \text{append}(T_{user}, args)$.

- if $cmd = \text{revoke-role}$, let $T_{role} \leftarrow \text{delete}(T_{role}, args)$ and $T_{user} \leftarrow \text{delete}(T_{user}, \{ \dots, (n_i, \cdot), \dots \})$, where n_i is the removed role included in $args$.
- if $cmd = \text{revoke-user}$, $T_{user} \leftarrow \text{delete}(T_{user}, args)$. Note that once a user is revoked, subsequent `ReadData` requests from the user will no longer satisfy the policy check. Therefore, the user cannot leverage his access to newly computed results or re-encrypted data related to on-chain data.

Then, update $\mathcal{L}'_P \leftarrow (T_{role}, T_{user})$ and send (`''admin''`, $sid, args, cmd, \mathcal{L}'_P, \bar{p}k_M$) to $\mathcal{S}$ and Cloud Server CS . Next, await (`''finished''`, sid, b) from $\mathcal{S}$ and if $b = 0$: return $(b, \perp)$. Else, update $\mathcal{L}_P \leftarrow \mathcal{L}'_P$ and return $(b, \mathcal{L}_P)$.

// $(v_i, DO_i) \in \mathcal{D}_i^P$, where v_i is the update secret for DO_i .

Upon receiving (`''WriteData''`, $sid, \mathcal{C}, \mathcal{D}_i^P$) from party P , send (`''enc-req''`, $sid, \mathcal{D}_i^P$) to $\mathcal{S}$, receive b from $\mathcal{S}$. Then, if $b = 1$, $\mathcal{L}_{data}[DO_i] = \text{append}(\mathcal{L}_{data}[DO_i], v_i)$ (for each $(v_i, DO_i) \in \mathcal{D}_i^P$) and return success; else return failed.

Upon receiving (`''ReadData''`, $sid, args$) from data user DU_i , if $args$ is illegal, return an error and terminate; else, parse $args \rightarrow (n, f, D)$. Then, if $(n, f, D) \notin T_{role} \vee (n, DU_i) \notin T_{user}$, return $\perp$. Next, for each $DO_j \in S$, retrieve all $\mathcal{L}_{data}[DO_j]$ to $\mathcal{D}$. After receiving b from $\mathcal{S}$, assert $b = 1$ and compute $(res, \pi) \leftarrow f(\mathcal{D})$. Finally, send (`''computed''`, $sid, l(res), \pi$) to CS , where $l(res)$ is the length of res , and send (`''computed''`, sid, res, π) to DU_i .

Security Features: We demonstrate why $\mathcal{F}_{\text{MISC}}$ achieves these security and privacy concepts.

- **Confidentiality:** The functionality $\mathcal{F}_{\text{MISC}}$ safeguards confidentiality by allowing only verified users to use the

operation function f to compute results res on dataset D . Authentication of data user DU_i is verified before the execution of f , and the re-encryption key remains undisclosed to the cloud server CS during new user permission addition via the “Admin” interface.

- **Computation integrity:** $\mathcal{F}_{\text{MISC}}$ first verifies parameter receipt, legality of requests, and the applicability of administration commands and arguments. It ensures parameters are accepted only after confirmation from $\mathcal{S}$ and that operations proceed with updated, not outdated, policy tables. The identity of request senders is authenticated, and administration commands are for specific actions like adding or removing users/roles. Arguments must comply with restrictions ensuring the legitimacy of operation functions and roles (for adding roles: $args = (n, f, S)$, $f \in \mathbb{F}$; for adding users: $args = (n, u)$, $n \in \mathbb{N}$), with legality checks at the start of “Admin” calls. Also, CS provides a computation proof π affirming that result res accurately derives from executing operation function f on a secret set D , in alignment with DU_i ’s policy. This proof, appended to res , is part of “ReadData”, ensuring policy compliance and validating computation correctness.
- **User revocation:** The functionality $\mathcal{F}_{\text{MISC}}$ supports revocation through the manager M ’s use of the “Admin” interface. In particular, revoking a user removes the corresponding authorization entry from T_{user} , so that subsequent “ReadData” requests from that user will fail the policy check. This guarantee is forward-looking; it prevents the revoked user’s future accesses to on-chain data, but does not retroactively invalidate outputs that were already obtained before revocation.

Let us consider a protocol, denoted as Π , which utilizes global functionalities including the global ledger $\mathcal{G}_L$ and trusted execution environment attestations $\mathcal{G}_{att}$. The protocol is said to securely realize the ideal functionality, $\mathcal{F}_{\text{MISC}}$ if the environment $\mathcal{E}$ cannot distinguish the interaction with the protocol Π in the *real* world from the interaction with the ideal functionality $\mathcal{F}_{\text{MISC}}$ in the *ideal* world. More precisely, the UC-realization of our ideal functionality, $\mathcal{F}_{\text{MISC}}$, is considered with respect to the global functionalities $(\mathcal{G}_L, \mathcal{G}_{att})$. In the real world, the ensemble of outputs from the environment $\mathcal{E}$, while interacting with the protocol Π and an adversary $\mathcal{A}$, is denoted by $\text{EXEC}_{\Pi, \mathcal{A}, \mathcal{E}}$. In the ideal world, parties convey their inputs to the ideal functionality $\mathcal{F}_{\text{MISC}}$. The output generated from the interaction of the environment $\mathcal{E}$ with $\mathcal{F}_{\text{MISC}}$, facilitated by the simulator $\mathcal{S}$, is represented as $\text{EXEC}_{\mathcal{F}_{\text{MISC}}, \mathcal{S}, \mathcal{E}}$. Consequently, we define the security of protocol Π as follows:

Definition 4.1 (GUC-security of Π): A protocol Π UC-realizes the ideal functionality $\mathcal{F}_{\text{MISC}}$ with respect to global functionalities $(\mathcal{G}_L, \mathcal{G}_{att})$, if for any PPT adversary $\mathcal{A}$, there exists a simulator $\mathcal{S}$, such that for any $\mathcal{E}$, the ensembles $\text{EXEC}_{\Pi, \mathcal{A}, \mathcal{E}}$ and $\text{EXEC}_{\mathcal{F}_{\text{MISC}}, \mathcal{S}, \mathcal{E}}$ cannot be distinguished.

V. DT-HPRE SCHEME

A. Definitions

We introduce a cryptographic primitive called the Double Trapdoor Homomorphic Proxy Re-encryption (DT-HPRE) scheme, which we later use to construct the MISC scheme.

Definition 5.1: A Homomorphic Proxy Re-encryption (HPRE) scheme $\Phi = (\text{KeyGen}, \text{Dec}, \text{Enc}, \text{ReEnc}, \text{ReKeyGen})$ with message space S is composed of:

- $(pk, sk) \leftarrow \text{KeyGen}(\lambda)$ computes a key pair (sk, pk) to decrypt and encrypt messages.
- $rk_{i \rightarrow j} \leftarrow \text{ReKeyGen}(sk_i, pk_j)$ outputs the re-encryption key $rk_{i \rightarrow j}$ for transferring the ciphertext from the key pk_i to pk_j .
- $c \leftarrow \text{Enc}(pk, m)$ encrypts the messages m under the public key pk and outputs the ciphertext c .
- $c' \leftarrow \text{ReEnc}(c, rk_{i \rightarrow j})$ re-encrypts the ciphertext c from the public key pk_i to pk_j .
- $m \leftarrow \text{Dec}(c, sk)$ decrypts the ciphertext c to m .

The DT-HPRE scheme inherits the definition of the HPRE scheme and satisfies the *correctness*, *single-hop*, and *unidirectional* properties and security.

Definition 5.2 (Unidirectional property): A DT-HPRE scheme is unidirectional [40] if for all re-encryption key $rk_{i \rightarrow j}$, the scheme only allows the transformation from pk_i to pk_j . That is, for every PPT adversary $\mathcal{A}$, there is a negligible function negl such that:

$$\begin{aligned} & \Pr[(pk_i, sk_i)/(pk_j, sk_j) \leftarrow \text{KeyGen}(\lambda), \\ & \quad rk_{i \rightarrow j} \leftarrow \text{ReKeyGen}(sk_i, pk_j), c \leftarrow \text{Enc}(pk_j, m), \\ & \quad c' \leftarrow \mathcal{A}(pk_j, c, rk_{i \rightarrow j}) : m = \text{Dec}(sk_i, c')] \leq \text{negl}(\lambda). \end{aligned} \quad (1)$$

Definition 5.3 (Single-hop property): A DT-HPRE scheme is single-hop [40] meaning that for every PPT adversary $\mathcal{A}$ there is a negligible function negl such that:

$$\begin{aligned} & \Pr[(pk_i, sk_i)/(pk_j, sk_j)/(pk_l, sk_l) \leftarrow \text{KeyGen}(\lambda), \\ & \quad rk_{i \rightarrow j} \leftarrow \text{ReKeyGen}(sk_i, pk_j), rk_{j \rightarrow l} \leftarrow \text{ReKeyGen}(sk_j, pk_l), \\ & \quad c \leftarrow \text{Enc}(pk_i, m), c' \leftarrow \text{reEnc}(c, rk_{i \rightarrow j}), \\ & \quad c'' \leftarrow \mathcal{A}(pk_j, c, c', rk_{j \rightarrow l}) : m = \text{Dec}(sk_l, c'')] \leq \text{negl}(\lambda). \end{aligned} \quad (2)$$

Definition 5.4 (Correctness): A unidirectional DT-HPRE scheme is perfectly correct concerning the space S if:

- For all key pairs (pk, sk) output by KeyGen and all messages $m \in D$, it has that: $\Pr[\text{Dec}(sk, \text{Enc}(pk, m)) = m] = 1$;
- For key pairs $(pk_1, sk_1) \cdots (pk_n, sk_n)$ (where $n > 1$) output by KeyGen , all key $rk_{i \rightarrow j}$ (where $i, j \leq n$) generated by $\text{ReKeyGen}(sk_i, pk_j)$, and all $m \in D$, it has formula: $\Pr[\text{Dec}(sk_j, \text{ReEnc}(\text{Enc}(pk_i, m), rk_{i \rightarrow j})) = m] = 1$.

Definition 5.5 (Security game): Let λ be the security parameter. The security game of the DT-HPRE scheme consists of an execution of a PPT adversary $\mathcal{A}$ with the following oracles:

- *Uncorrupted key generation oracle:* On request, the oracle generates a fresh key pair $(pk, sk) \leftarrow \text{KeyGen}(\lambda)$ and provides pk to the adversary $\mathcal{A}$.
- *Corrupted key generation oracle:* On request, the oracle generates a fresh key pair $(pk, sk) \leftarrow \text{KeyGen}(\lambda)$ and provides both pk and sk to the adversary $\mathcal{A}$.
- *Re-encryption key generation oracle $\mathcal{O}_{rkey}$:* On input (pk_i, pk_j) from $\mathcal{A}$, where pk_i and pk_j were previously generated by the key generation oracle, the re-encryption key $rk_{i \rightarrow j} \leftarrow \text{ReKeyGen}(sk_i, pk_j)$ is returned, subject to the conditions: both pk_i and pk_j are either corrupted or uncorrupted, or pk_i is corrupted, and pk_j is honest.
- *Re-encryption oracle $\mathcal{O}_{renc}$:* On input (pk_i, pk_j, c) , where pk_i and pk_j were previously generated by the key

generation oracle, the oracle returns a re-encrypted ciphertext $c' \leftarrow \text{ReEnc}(rk_{i \rightarrow j}, c)$, unless pk_i is corrupted and pk_j is uncorrupted, in which case $\perp$ is returned.

- *Challenge oracle:* This oracle can be queried only once. On input (pk^*, m_0, m_1) , where pk^* is the designated challenge key, the oracle randomly selects a bit $b \in \{0, 1\}$ and returns the challenge ciphertext $c^* \leftarrow \text{Enc}(pk^*, m_b)$. If pk^* is corrupted, $\perp$ is returned.
- *Decision oracle:* This oracle can be queried only once. On input b' , the oracle outputs 1 if $b' = b$ and pk^* remains uncorrupted. Otherwise, it outputs 0.

We say that the adversary $\mathcal{A}$ wins the game with advantage $\text{Adv}_{\text{DT-HPRE}, \mathcal{A}} = \text{negl}(\lambda)$ (where negl is a negligible function) if the probability $\Pr[\text{Sec}_{\mathcal{A}}(\lambda)]$ —over the random choices made by $\mathcal{A}$ and the oracles—that the decision oracle outputs 1 when invoked is at most $1/2 + \text{negl}(\lambda)$.

Definition 5.6 (Security): A DT-HPRE scheme is secure for domain S of messages if any PPT adversary wins the game defined in Definition 5.5 only with negligible advantage.

B. Construction

Overview: The key idea of our DT-HPRE scheme involves a key pair $(sk, pk = g^{sk})$ and the HPRE scheme $\Psi = (\text{KeyGen}, \text{ReKeyGen}, \text{Enc}, \text{ReEnc}, \text{Dec})$ [41] which supports homomorphic addition and proxy re-encryption but is unable to provide double trapdoors for ciphertext decryption. To bridge this gap, we propose a modular construction that augments a standard HPRE scheme with an additional ElGamal-style trapdoor. The core of this design is a specially structured ciphertext: $c = \text{Enc}(m, pk) = (m \cdot pk^{rand}, \Psi.\text{Enc}(rand))$. This ciphertext c can be decrypted using the secret key sk , or alternatively, one can use the HPRE scheme's decryption key to obtain the randomness $rand$ and then extract the message m from c . This approach enables double-decryption trapdoors, allowing decryption via two keys. Further, by designing ciphertexts with a homomorphic algebraic structure and leveraging the homomorphic and re-encryption properties of the underlying HPRE scheme Ψ , our DT-HPRE scheme supports both homomorphic operations and proxy re-encryption.

Construction 5.1 (DT-HPRE): Let λ be the security parameter. We generate an RSA modulus $n = pq$, where p and q are two λ -bit primes. Let $\mathbb{G}$ denote the multiplicative group $\mathbb{Z}_{n^2}^*$. Select a generator $g \in \mathbb{G}$ such that g has maximal order. Besides, the order of the group follows $\text{ord}(\mathbb{G}) = \phi(n^2)/2$. We define the message space as $S = \mathbb{Z}_n$. Our underlying HPRE scheme is instantiated using the construction from [41] $\Psi = (\text{KeyGen}, \text{ReKeyGen}, \text{Enc}, \text{ReEnc}, \text{Dec})$. The message space of Ψ can be $\mathbb{Z}_{\text{ord}(\mathbb{G})}$, which aligns with the exponent space of our secondary trapdoor. Based on these parameters and Ψ , our construction Φ is defined as follows.

- **KeyGen:** run $(pk_{1,u}, sk_{1,u}) \leftarrow \Psi.\text{KeyGen}(\lambda)$, sample a random $sk_{2,u} = x \in [1, \text{ord}(\mathbb{G})]$, generate $pk_{2,u} = g^x \bmod n^2$, and return $(pk_u := (pk_{1,u}, pk_{2,u}), sk_u := (sk_{1,u}, sk_{2,u}))$.
- **ReKeyGen:** Given sk_i, pk_j , generate re-encryption key $rk_{i \rightarrow j} \leftarrow \Psi.\text{ReKeyGen}(sk_{1,i}, pk_{1,j})$ and return $rk_{i \rightarrow j}$.
- **Enc:** Given the key pk_u and message m , parse the key pk_u as $(pk_{1,u}, pk_{2,u})$, sample $r \in \mathbb{Z}_{\text{ord}(\mathbb{G})}$, and compute $e_1 = [(1 + n \cdot m) \cdot pk_{2,u}^r] \bmod n^2$, $e_2 = g^r \bmod n^2$, and $e_3 = \Psi.\text{Enc}(pk_{1,u}, r)$, set and return ciphertext $c = (e_1, e_2, e_3)$.

- **ReEnc**: Given a ciphertext c and re-encryption key $rk_{i \rightarrow j}$, parse c as (e_1, e_2, e_3) , sample $r' \in \mathbb{Z}_{\text{ord}(\mathbb{G})}$, compute $e'_1 = e_1 \cdot pk_{2,j}^{r'}$, $e'_2 = e_2 \cdot g^{r'}$, and $e'_3 \leftarrow \Psi.\text{ReEnc}(rk_{i \rightarrow j}, e_3 \oplus \Psi.\text{Enc}(pk_{1,u}, r'))$, set $c' = (e'_1, e'_2, e'_3)$, and return a re-encrypted ciphertext c' .
- **Dec₁**: Given a ciphertext c and the first secret key $sk_{1,u}$, parse c as (e_1, e_2, e_3) , compute $m' = \Psi.\text{Dec}(e_3, sk_{1,u})$, compute $m = L(e_1 / pk_{2,u}^{m'} \bmod n^2)$ (where $L(u) = (u - 1)/n$), and return m .
- **Dec₂**: Given a ciphertext c and the second secret key $sk_{2,u}$, parse c as (e_1, e_2, e_3) , compute and return $m = L(e_1 / e_2^{sk_{2,u}} \bmod n^2)$.

Remark: Note that our DT-HPRE construction is generic in nature. Any other HPRE scheme Ψ can be used to construct a DT-HPRE scheme if it satisfies three criteria: 1) it must support additive homomorphism to maintain the overall algebraic structure; 2) it must be unidirectional and single-hop to ensure controlled data flow; and 3) it must operate over a group structure where its message space aligns with the exponent space of our secondary trapdoor.

C. Security Analysis

We state the main security theorem as follows.

Theorem 5.1. Assume that the DDH assumption holds in the group $\mathbb{Z}_{n^2}^*$ [16] and let the HPRE scheme Ψ be an IND-CPA secure, unidirectional, and single-hop HPRE, then Construction 5.1 is a unidirectional, single-hop, correct, and secure DT-HPRE scheme.

Proof: The security proof of Theorem 5.1 is as follows.

Single-hop property: It straightforwardly follows from the single-hop property of the HPRE scheme Ψ .

Unidirectional property: It directly inherits the unidirectional property of the HPRE scheme Ψ .

Correctness: Based on the above construction of a DT-HPRE scheme, we describe the correctness as follows:

Initial ciphertext: For all key pairs (pk, sk) (where $pk = (pk', pk'')$ and $sk = (sk', sk'')$) output by KeyGen and all messages $m \in D$, there are two different decryption phases. For the partial decryption key sk'' , it has that:

$$\begin{aligned}
 m &= \text{Dec}(sk, \text{Enc}(pk, m)) \\
 &= \text{Dec}(sk, (e_1 = (1 + n \cdot m) * pk''^{mr}, e_2 = g^r, \\
 &\quad e_3 = \Psi.\text{Enc}(pk', r))) \\
 &= L(e_1 / pk''^{(\Psi.\text{Dec}(e_3, sk'))} \bmod n^2) \\
 &= L((1 + n \cdot m) * pk''^{mr} / pk''^{mr}, \bmod n^2) \\
 &= L((1 + n \cdot m)) = m
 \end{aligned} \tag{3}$$

Similarly, for the partial decryption key sk' , it has that:

$$\begin{aligned}
 m &= \text{Dec}(sk, \text{Enc}(pk, m)) \\
 &= \text{Dec}(sk, (e_1 = (1 + n \cdot m) * pk''^{mr}, e_2 = g^r, \\
 &\quad e_3 = \Psi.\text{Enc}(pk', pk''^{mr}))) \\
 &= L(e_1 / e_2^{sk''}, \bmod n^2) \\
 &= L((1 + n \cdot m) * pk''^{mr} / pk''^{mr}, \bmod n^2) \\
 &= L((1 + n \cdot m)) = m.
 \end{aligned} \tag{4}$$

Re-encrypted ciphertext: The algorithm ReEnc enables users to use the re-encryption key $rk_{i \rightarrow j}$ to re-encrypt the initial ciphertext $c_1 = (e_1, e_2, e_3)$ under the public key pk_i to the ciphertext c_2 under the public key pk_j . The re-encrypted ciphertext $c_2 = (e'_1, e'_2, e'_3)$ is generated as follows:

$$\begin{aligned}
 (e'_1, e'_2, e'_3) &= (e_1 \cdot pk_{2,j}^{r'}, e_2 \cdot g^{r'}, \Psi.\text{ReEnc}(rk_{i \rightarrow j}, e_3 \\
 &\quad \oplus \Psi.\text{Enc}(pk_{1,u}, r')))
 \end{aligned} \tag{5}$$

Based on the decryption key sk_j , the decryption of the re-encrypted ciphertext c_2 is as follows:

$$\begin{aligned}
 m &= \text{Dec}(sk_j, \text{Enc}(pk, m)) \\
 &= \text{Dec}(sk_j, (e'_1 = (1 + n \cdot m) * pk_i^{r+r'}, e'_2 = g^{r+r'}, e'_3)) \\
 &= L(e_1 / pk_i^{(\Psi.\text{Dec}(e'_3, sk'_j))} \bmod n^2) = L((1 + n \cdot m)) = m.
 \end{aligned} \tag{6}$$

Homomorphic Addition: Assume two ciphertexts (c_1, c_2) are under the public key $pk = (pk', pk'')$.

$$\begin{aligned}
 c_1 &= (e_1, e_2, e_3) = ((1 + n \cdot m_1) * pk''^{r_1}, g^{r_1}, e_3) \\
 c_2 &= (e'_1, e'_2, e'_3) = ((1 + n \cdot m_2) * pk''^{r_2}, g^{r_2}, e'_3) \\
 c' &= c_1 \oplus c_2 = (e_1 * e'_1, e_2 * e'_2, e_3 * e'_3) \\
 &= ((1 + n \cdot (m_1 + m_2)) * pk''^{(r_1+r_2)}, g^{(r_1+r_2)}, e_3 \oplus e'_3).
 \end{aligned} \tag{7}$$

Security Analysis: We target IND-CPA security for MISC, as the adversary's capabilities are effectively restricted to the CPA model. Under our protocol, re-encryption keys are isolated within a TEE, preventing any exposure to the untrusted Cloud Server. Since the Server is limited to observing ciphertexts and interacting with public parameters—without the ability to perform unauthorized decryption or key-derivation—the CPA model accurately captures the threat environment. Therefore, IND-CPA security for the DT-HPRE primitive provides sufficient protection within our integrated framework. Next, we define a sequence of hybrid experiments to formally prove that no polynomial-time adversary can break the security of our construction with non-negligible advantage. We assume all messages from the adversary are well-formed, leading to an immediate abort otherwise. Formally, $\Pr[\text{Hyb}_i(\lambda) = 1]$ is defined as the possibility of the experiment i outputting 1.

Hyb₀: This is in line with the Construction 5.1.

Hyb₁: Same as Hyb₀, except that the challenge oracle replaced the third part of the challenge ciphertext e_3 and its re-encrypted ciphertext e'_3 via the simulator Sim_{HPRE} of the selected HPRE scheme [41]. The adversaries query the decision oracle via their guess b on messages m_b and return the oracle outputs.

Hyb₂: Same as Hyb₁, except that the challenge oracle computed the first part of the challenge ciphertext via the formula: $e_1 = (1 + n \cdot m) * g_0$, where $g_0 = g^z$ is randomly chosen from $\mathbb{G}$. Note that $pk_2^r = g^{ar}$.

Claim 5.1. $\{\text{Hyb}_0(\lambda)\}_{\lambda \in \mathbb{N}} \approx_c \{\text{Hyb}_1(\lambda)\}_{\lambda \in \mathbb{N}}$.

Proof: The proof of this claim can be directly derived from the security of the HPRE scheme. That is, the adversary can not distinguish the experiments $\{\text{Hyb}_0(\lambda)\}$, $\{\text{Hyb}_1(\lambda)\}$, meaning the following formula holds:

$$|\Pr[\text{Hyb}_0(\lambda) = 1] - \Pr[\text{Hyb}_1(\lambda) = 1]| \leq \text{negl}(\lambda). \tag{8}$$

□

Claim 5.2. $\{\text{Hyb}_1(\lambda)\}_{\lambda \in \mathbb{N}} \approx_c \{\text{Hyb}_2(\lambda)\}_{\lambda \in \mathbb{N}}$.

Proof: In the experiment Hyb_2 , the challenge ciphertext c^* received by the adversary $\mathcal{A}$ is independent of the messages m_b obtained. Thus, the possibility of the decision oracle queried by $\mathcal{A}$ outputting 1 is as follows:

$$\Pr[\text{Hyb}_2(\lambda) = 1 - 1/2] \leq \text{negl}(\lambda). \quad (9)$$

Under the DDH assumption, $\mathcal{A}$ cannot distinguish the tuple $(e_1 = (1 + n \cdot m) * pk^{rr}, e_2 = g^r, e_3)$ and $(e_1 = (1 + n \cdot m) * g_0, e_2 = g^r, e_3)$. That is, the challenge ciphertext from $\text{Hyb}_2(\lambda)$ is identically distributed to the challenge ciphertext from $\text{Hyb}_1(\lambda)$, meaning the following holds:

$$|\Pr[\text{Hyb}_1(\lambda) = 1] - \Pr[\text{Hyb}_2(\lambda) = 1]| \leq \text{negl}(\lambda). \quad (10)$$

Based on the (8), (9), and (10), we have that:

$$\begin{aligned} \text{Adv}_{\text{DT-HPRE}, \mathcal{A}} &= \Pr[\text{Sec}_{\mathcal{A}}(\lambda)] = |\Pr[\text{Hyb}_2(\lambda) = 1] - 1/2| \\ &= |\Pr[\text{Hyb}_2(\lambda) = 1] - \Pr[\text{Hyb}_1(\lambda) = 1]| \\ &= |\Pr[\text{Hyb}_2(\lambda) = 1] - \Pr[\text{Hyb}_0(\lambda) = 1]| \\ &\leq \text{negl}(\lambda). \end{aligned} \quad (11)$$

To summarize, Theorem 5.1 holds in the Construction 5.1. Note that following [40], the game-based security of PRE can imply the weakly UC security. $\square$

VI. CONCRETE DESIGN

This section introduces the design of MISC, a TEE-assisted flexible data management system tailored for PSCs. By leveraging cloud server's TEEs, data owners can securely delegate management responsibilities over their on-chain data to managers in varying privilege levels.

A. PSC Integration Assumptions

We assume that PSC systems adopt the DT-HPRE scheme Φ (cf. Section V) as their underlying encryption mechanism. This enables PSC callers to operate on “foreign data securely”—i.e., data owned by parties other than the caller. Specifically, during the initial PSC development and setup phase, data owners collaborate with the cloud server's TEE to generate their cryptographic key pairs (pk_P, sk_P) via the $\Phi.\text{KeyGen}$ algorithm. The secret key sk_P is structured with two components: 1) a common trapdoor $sk_{2,P}$, managed by the TEE, and 2) a unique trapdoor $sk_{1,P}$, held by the data owner. Once PSCs are deployed, a data owner P can execute the PSC's functions to update their on-chain data. To ensure the confidentiality of sensitive information v during this process, the owner first encrypts it using their public key pk_P via the encryption algorithm $\Phi.\text{Enc}$, resulting in the ciphertext $val = \Phi.\text{Enc}(pk_P, v)$, which is then uploaded to the PSC.

B. Design Overview

To build upon our data management on PSCs, we introduce two new components in MISC to govern access to on-chain data (refer to Section III-A): the Access Control Module empowers managers (whether they are privileged or constrained) to securely define and manage user permissions; and the Data Access Flow leverages these established permissions, providing

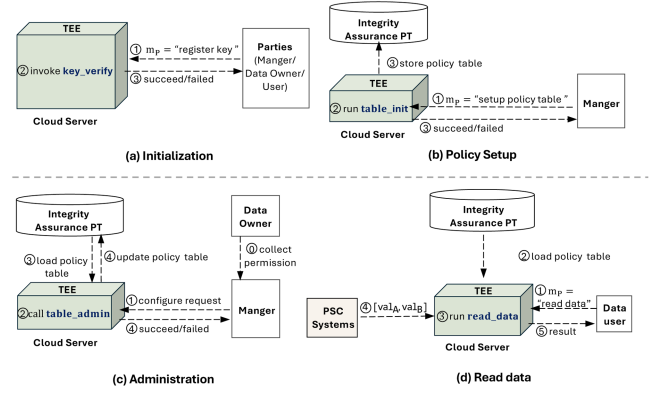


Fig. 2. System workflows showing key sub-procedures. (a)–(c) Access Control Module: (a) Secure registration and verification of party keys within the TEE. (b) The manager initializes and stores Integrity Assurance Policy Tables (PTs) via the TEE. (c) The manager updates existing permissions in the PTs based on data owner approval. (d) Data access flow: The TEE verifies a user's request against loaded PTs, retrieves data from PSC systems, and returns the result.

formalized interfaces for users to access authorized data or obtain computation results.

Access Control Module: In this module, the designated manager M in MISC (serving as the access control manager), with the help of the cloud server's TEEs, configures Integrity Assurance Policy Tables (PTs) to distribute and maintain user permissions associated with PSCs' on-chain data, with the approval of data owners. As illustrated in Fig. 2(a)–(c), the module generally involves three components: (a) *initialization*: registering key materials, e.g., data users' encryption public key and signature verification key, (b) *policy setup*: establishing PTs, including user-role and role-permission tables to define granular user permissions and (c) *administration*: managing user permissions by updating the associated permissions in PTs as access requirements evolve. By shifting access control operations off-chain into TEEs, MISC significantly reduces gas consumption. By shifting access control operation off-chain into TEEs, MISC reduces gas consumption. This key feature enables the following forms of robust and differentiated access control without requiring fundamental changes to the underlying blockchain:

- **Privileged manager:** A privileged manager can leverage the common trapdoor $sk_{2,P}$ in the TEE to design user permissions (i.e., *decryption and computation permissions* within the TEE) in PTs. This manager assigns users permissions to collect complex statistical computation results over the plaintext of on-chain data since the common trapdoor can decrypt the on-chain data within the TEE.
- **Constrained manager:** A constrained manager is limited to defining user permissions for *re-encryption operations*. Unlike the privileged manager, the constrained manager cannot instruct the TEE to decrypt on-chain data for complex statistical computations directly. Instead, they rely exclusively on re-encryption keys $rk_{P \rightarrow U}$ that are generated and authorized by the data owner P for a specific user U (not by the manager) and are provided to the system only in encrypted form $c_{rk} = \text{Enc}(pk_{att}, rk_{P \rightarrow U})$. These encrypted re-keys c_{rk} are securely stored and used inside the TEE (decryptable only with sk_{att}), while the

constrained manager only references c_{rk} when configuring permissions in PTs, enabling specific users to access re-encrypted ciphertexts that they can decrypt with their private keys.

Data Access Flow: The data access flow (cf. Fig. 2(d)) of MISC leverages TEEs to regulate user access to on-chain data, strictly following permissions defined by managers in PTs. Depending on the privilege level of the manager, the access process and the final data obtained by users significantly differ: 1) *Under privileged manager policies:* After the TEE verifies the user's permissions against PTs, it directly decrypts the target data using the common trapdoor ($sk_{2,P}$). The TEE then securely performs computations authorized by the permissions and provides users with the computation results. 2) *Under constrained manager policies:* After permission verification, the TEE re-encrypts the target ciphertext for user U using specific re-encryption keys ($rk_{P \rightarrow U}$) for user U . The user U subsequently employs their private key to decrypt this re-encrypted ciphertext.

Permission Definition: Following the Role-Based Access Control (RBAC) model proposed by Liu et al. [37], the PTs in MISC include: a user-role assignment table, denoted as T_{user} , and a role-permission assignment table, denoted as T_{role} . A core RBAC system consists of sets of users U , roles R , permissions P , and a subset of on-chain data D . $T_{user} \subseteq U \times R$ defines associations between users and their roles, whereas $T_{role} \subseteq R \times P$, specifies which permissions are associated with each role. Under this model, a user $u \in U$ acquires permission $p \in P$ if and only if there exists a role $n \in R$ such that $(u, n) \in T_{user}$ and $(n, p) \in T_{role}$. Each permission p is defined as a tuple $p := (D, f, \Sigma, mtl)$, where D is the subset of on-chain data, f denotes a computation function, Σ comprises approvals provided by data owners, and mtl is the privilege level of the manager. Consequently, the permission assignment (n, p) indicates that role n has been explicitly authorized via approvals Σ to access results produced by applying the computation function f to data elements in the set D .

C. Access Control Module

The access control module and data access flow rely on several cryptographic schemes: the DT-HPRE scheme Φ for encrypting on-chain messages; the PKE scheme for encrypting re-encryption keys; the NIZK scheme Ξ for verifying the correctness of key materials; and the signature scheme Ω for signing interactive messages among parties. To illustrate the module, consider the following scenario with two parties: Alice, who plays the role of data user, and Bob, who plays the role of data owner. Based on the data access permissions assigned by a manager, Alice accesses Bob's on-chain secrets with the help of the cloud server's TEE. We assume each sender, including Alice, Bob, managers, and TEEs, signs *all* interactive messages, and we omit the details of signature generation and verification for brevity.

Phase I. Initialization: Following the programs `progs` installation into the cloud server's TEE by the PSC developer (refer to Algorithm 1), its global state counters are initialized. These include γ_t , designed to ensure that TEEs always load the latest PTs, and γ_c , intended to safeguard administrative commands against replay attacks. To ensure administrative commands are fresh and to prevent replay attacks, managers must also locally initialize and maintain a corresponding command counter, denoted as γ'_c .

Algorithm 1: The Programs `progs` Installed Into the TEE.

```

1 Hardware keys: ( $sk_{att}, pk_{att}$ ); Global counters:  $\gamma_c, \gamma_t$ ; Manager
  privilege level:  $mtl \in \{\text{privileged}, \text{constrained}\}$ ;
2 User key map:  $\text{key}[\cdot]$  (an internal, persistent map, initially empty,
  storing verified user public keys ( $pk, \overline{pk}$ )); Global ledger:  $\mathcal{G}_L$ ;
3 Function key_verify ( $pk_P, \overline{pk}_P, \pi_{sk}$ )
4   if  $\exists \text{Vry}(\pi_{sk}, (pk_P, \overline{pk}_P)) = 1$  then
5      $\text{key}[P] \leftarrow (pk_P, \overline{pk}_P)$ ;  $\sigma_{att} \leftarrow \Omega.\text{Sign}(sk_{att}, \text{key})$ ;
6     return (succeed, key,  $\sigma_{att}$ );
7   return failed;
8 Function table_init ( $T_{user}, T_{role}, \sigma_M, \text{key}$ ):
9   if  $\Omega.\text{Verify}(\text{key}[M], \sigma_M, (T_{user}, T_{role})) = 1$  then
10     $\gamma_t \leftarrow \gamma_t + 1$ ;  $\sigma_{att} \leftarrow \Omega.\text{Sign}(sk_{att}, (\gamma_t, \gamma_c, T_{role}, T_{user}))$ ;
11    return (succeed,  $\sigma_{att}, T_{role}, T_{user}$ );
12   return failed;
13 Function
  table_admin ( $\gamma'_c, \sigma_M, T_{role}, T_{user}, cmd, args, mtl, \sigma_{att}$ ):
14   if  $\gamma_c \neq \gamma'_c \vee \Omega.\text{Verify}(\text{key}[M], \sigma_M, (args, \gamma'_c)) \neq 1 \vee$ 
     verifyAtt( $\sigma_{att}, pk_{att}, (\gamma_t, \gamma_c, T_{role}, T_{user})) \neq 1$  then
15     return failed;
16   if cmd = add_role then
17     /* Add role  $n$  with permissions  $p = (D, f, \Sigma)$ . */
18     foreach  $\sigma_B \in \Sigma$  do
19       if  $\Omega.\text{Verify}(\overline{pk}_i, \sigma_B, (args, mtl, pk_m)) \neq 1$  then
20         return ( $T_{role}$ , failed);
21      $T_{role} \leftarrow \text{add}(T_{role}, args, mtl)$ ;
22   else if cmd = grant_user then
23     if  $args.n \notin T_{role}$  then
24       return failed;
25      $mtl \leftarrow \text{retrieve}(n, T_{role})$ ;
26     if  $mtl = \text{privileged}$  then
27        $T_{user} \leftarrow \text{add}(T_{user}, (args.n, args.u))$ ;
28     else if  $mtl = \text{constrained}$  then
29        $rk_{B \rightarrow A} \leftarrow \text{Dec}_{sk_{att}}(args.c_{rk})$ ;
30       if
31          $\exists \text{Vry}(args.\pi_{rk}, (rk_{B \rightarrow A}, args.pk_A, args.pk_B)) \neq 1$ 
32         then
33         return failed;
34        $T_{user} \leftarrow \text{add}(T_{user}, (args.n, args.u, args.c_{rk}))$ ;
35   else if cmd = revoke_role then
36      $T_{role} \leftarrow \text{delete}(T_{role}, args.n)$ ;
37      $T_{user} \leftarrow \text{delete}(T_{user}, (args.n, \cdot))$ ;
38   else if cmd = revoke_user then
39      $T_{user} \leftarrow \text{delete}(T_{user}, args.u)$ ;
40   else
41     return failed;
42    $\gamma_c \leftarrow \gamma_c + 1$ ;  $\gamma_t \leftarrow \gamma_t + 1$ ;
43    $\sigma_{att} \leftarrow \Omega.\text{Sign}(sk_{att}, (\gamma_t, \gamma_c, T_{role}, T_{user}))$ ;
44   return (succeed,  $\gamma_c, \gamma_t, T_{role}, T_{user}$ );
45 Function read_data ( $T_{role}, T_{user}, u, \sigma_{att}$ ):
46   if verifyAtt( $\sigma_{att}, pk_{att}, (\gamma_t, \gamma_c, T_{role}, T_{user})) \neq 1$  then
47     return failed;
48   ( $n, c_{rk}$ )  $\leftarrow \text{retrieve}(u, T_{user})$ ;
49   if  $n = \perp$  then
50     return failed;
51   ( $f, D, mtl$ )  $\leftarrow \text{retrieve}(n, T_{role})$ ;  $\mathcal{D} \leftarrow \mathcal{G}_L.\text{Read}(D)$ ;
52   if  $mtl = \text{privileged}$  then
53      $v \leftarrow f(u, \mathcal{D}, sk_{2,P})$ ;  $res \leftarrow \Phi.\text{Enc}(v, pk_P)$ ;
54     /*  $f$  invokes  $\Phi.\text{Dec}_2$  to decrypt and then operate over  $\mathcal{D}$ . */
55   else if  $mtl = \text{constrained}$  then
56      $rk \leftarrow \text{Dec}_{sk_{att}}(c_{rk})$ ;
57      $res \leftarrow f(\mathcal{D}, rk)$  /*  $f$  invokes  $\Phi.\text{ReEnc}$  to re-encrypt data  $\mathcal{D}$ . */
58   return ( $res, \text{succeed}$ );

```

When issuing an administrative request, the manager includes their current γ'_c ; the TEE validates this against its internal γ_c before processing the command, ensuring both counters are consistent. These foundational steps—the TEE installation and full setup of the counter mechanisms—precede and enable the main activities of the Initialization phase.

Next, the initialization phase, as depicted in Fig. 2(a), primarily concerns the registration of key materials by involved parties (①-③). Party $P = \{\text{Alice, Bob, Manager}\}$ initiates key registration by sending message m_P (containing encryption public key $pk_P = (pk_{1,P}, pk_{2,P})$ and signature verification key $\overline{pk}_P$) along with a Zero-Knowledge Proof (ZKP) π_{sk} to the server's TEE. The ZKP π_{sk} demonstrates the validity of pk_P and $\overline{pk}_P$, allowing the TEE to verify public key correctness without revealing the private keys. Since $pk_{2,P}$ is the public key corresponding to the common trapdoor $sk_{2,P}$, which is already stored in the TEE, the proof verifying $(sk_P, pk_P) \in \Phi.\text{KeyGen}(\lambda)$ simplifies to establishing that $pk_{1,P} = g^{sk_{1,P}}$. Thus, the ZKP π_{sk} is defined as follows:

$$\pi_{sk} \leftarrow \Xi.\text{Prove}((sk_{1,P}, \overline{sk}_P), (\overline{sk}_P, \overline{pk}_P) \in \Omega.\text{KeyGen}(\lambda) \wedge pk_{1,P} = g^{sk_{1,P}}). \quad (12)$$

Upon receiving m_P and π_{sk} , the TEE invokes `key_verify`. Successful verification leads to storing verified keys internally as $\{key\{P\} \leftarrow (pk_P, \overline{pk}_P)\}$. Finally, the TEE issues a succeed message and a remote attestation, certifying the secure registration of P 's keys (Algorithm 1, lines 4-6).

Phase II. Policy Setup: This phase is executed only once for initializing PTs. Initially, the assignment tables are empty, represented as $[T_{role} \leftarrow \emptyset; T_{user} \leftarrow \emptyset]$. The manager initiates the setup by transmitting these assignment tables, along with a certification σ_M , to the server's TEE (step ① in Fig. 2(b)), where $\sigma_M \leftarrow \Omega.\text{Sign}(\overline{sk}_M, (T_{role}, T_{user}))$. Upon receiving these tables, the TEE invokes `table_init` function in Algorithm 1 to validate the manager's signature σ_M and verify the integrity of the received tables (②). During this operation, the TEE updates its internal counter γ_t , appends it to the tables being initialized, and subsequently signs the updated data using the attestation key sk_{att} (Algorithm 1, lines 10-11). Suppose the execution of `table_init` is successful. In that case, the TEE then provides a "succeed" message along with a remote attestation σ_{att} , certifying the state update and integrity of the PTs. Following this attestation, the cloud server locally records the initialized PTs (③). This establishes the required policy tables for subsequent access control operations.

Phase III. Administration: Fig. 2(c) shows how MISC adopts RBAC [37] to support flexible access administration. Let cmd be the selective administration command and $args$ be the required arguments to accomplish the selective command. Upon receiving cmd and $args$, along with a certification $\sigma_M = \Omega.\text{Sign}(\overline{sk}_M, (args, \gamma'_c))$ from the manager (①), the TEE invokes `table_admin` (②) and loads the PTs (③) for the following operations. The `table_admin` function (Algorithm 1, line 13) first performs preliminary checks (lines 14-15) to ensure the integrity and security of the administration command. These checks include validating the manager's certification σ_M , guaranteeing the internal counter γ_c matches the received counter γ'_c , and verifying the attestation σ_{att} . Specifically, the comparison involving γ_c is crucial: it ensures that previously used commands cannot be replayed by adversaries, as any discrepancy in the counters will invalidate the administration command. The TEE can ensure the loaded PTs are the latest version by verifying σ_{att} with γ_t . Once all verification succeeds, the TEE adjusts the PTs based on cmd as well as $args$, and returns the updated counter γ_c and "succeed" (④). Next, we further discuss how to obtain the required arguments for different administration commands.

1) $cmd=add_role, args=(n, D, f, \Sigma)$: This command adds a new role to the table T_{role} . The specific operations of the manager here are dependent on their privilege level:

- **Privileged manager:** The manager creates a new role n endowed with the computation function f . The function operates on the data items $D = (val_id_B \leftarrow val_B)$ owned by Bob. The manager needs to collect his/her trusted level $mtl = privileged$ (representing that Bob highly trusts the manager) and signatures $\Sigma = \{\sigma_B\}$ that were signed on $args, mtl$, and the manager's public key pk_m , from Bob. The signature on $args$ and pk_m indicates that Bob approves the role n to apply the function f on his on-chain dataset D , and the manager can be considered as *privileged* to manage Bob's on-chain data.
- **Constrained manager:** The constrained manager is only allowed to create a role n with the re-encryption function f . Similarly, the manager needs to collect his/her trusted level $mtl = constrained$ (meaning that Bob has less trust for the manager) and signatures $\Sigma = \{\sigma_B\}$ from Bob.

2) $cmd=grant_user, args=(n, Alice)/(n, Alice, c_{rk}, \pi_{sk})$:

This command facilitates assigning a predefined role n to a user, Alice, and subsequently updates this assignment in the table T_{user} . The operational procedure for this command is contingent upon the manager's designated privilege level:

- **Privileged manager:** To assign role n to Alice, the manager directly dispatches the $cmd = grant_user$ command, along with the arguments $args = (n, Alice)$, to the server's TEE.
- **Constrained manager:** Conversely, to assign role n to Alice, a constrained manager first obtains the encrypted re-encryption key c_{rk} , and its associated correctness proof, π_{sk} , from the data owner, Bob. The re-encryption key, $rk_{B \rightarrow A}$ (which is c_{rk} encrypted by pk_{att}), is used to transform Bob's on-chain data for Alice to decrypt. The proof π_{sk} is as follows:

$$\pi_{rk} \leftarrow \Psi.\text{Prove}((sk_B), \{rk_{B \rightarrow A} = pk_A^{1/sk_B} \wedge pk_B = g^{sk_B}\}). \quad (13)$$

This proof converts the statement regarding the re-encryption key—specifically, the relation $rk_{B \rightarrow A} = pk_A^{1/sk_B}$ —into its equivalent tuple representation $pk_A = rk_{B \rightarrow A}^{sk_B}$. Then, the manager prepares $args$ to include the role n , user *Alice*, c_{rk} , and the proof π_{rk} . The manager then invokes the `table_admin` function with $cmd = grant_user$ and these comprehensive $args$. Next, the function decrypts c_{rk} to obtain $rk_{B \rightarrow A}$ and validates this key using the provided proof π_{rk} (Algorithm 1, lines 29-31). If this verification succeeds, the `table_admin` function updates the T_{user} table with Alice's new role assignment.

3) $cmd=revoke_role/user, args=n/u$: To revoke a role entirely (including its assignments to users), the manager sends the $cmd = revoke_role$ command with $args$ containing the role name n (Algorithm 1, lines 33-34). To revoke a specific user, the manager sends the $cmd = revoke_user$ command with $args$ containing the user's identifier u (lines 35-36).

D. Data Access Flow

The data access flow, as illustrated in Fig. 2(d), outlines the interactions among data users, the cloud server, and the PSC

system. This process enables data users, such as Alice (denoted as user u), to read on-chain values according to their permissions, with the assistance of the cloud server. In this example, Alice initiates the process by submitting a data request to the cloud server (①). The TEE reloads the latest PTs (②) that store user u 's access permissions, and triggers the `read_data` function (③). The `read_data` function first verifies PT freshness via state counters γ_t and attestation (Algorithm 1, line 43). If successful, it retrieves user u 's role n and encrypted re-encryption key c_{rk} from T_{user} , along with target data fields D , operation function f , and manager privilege level mtl from T_{role} (lines 45–48). Next, `read_data` calls $\mathcal{G}_L$ to retrieve on-chain values D for fields D (line 48). The TEE's subsequent data processing strategy is contingent upon the manager's privilege level mtl :

- *Permission managed under privileged manager:* In this scenario, the function f processes the data $\mathcal{D}$ collected from the data fields D stored in the global ledger $\mathcal{G}_L$. This execution typically involves decrypting $\mathcal{D}$ using the common trapdoor $sk_{2,P}$ via the decryption algorithm $\Phi.\text{Dec}_2$, followed by performing the computations defined by f on the decrypted data (lines 49–50). The final result of the computation, v , is then encrypted using Alice's public key pk_P (④) (line 50).
- *Permission managed under constrained manager:* Alternatively, if the manager is at a low privilege level, the TEE first decrypts the user-specific encrypted re-encryption key c_{rk} (retrieved from T_{user}) using the key sk_{att} . This yields the re-encryption key rk (line 53). The data $\mathcal{D}$ is then processed by the function f utilizing rk . This processing involves explicitly re-encrypting the data using $\Phi.\text{ReEnc}$ (line 54). The re-encrypted result, res , is then returned.

Discussion. multi-TEE deployment: Although the above construction is presented with a single cloud server, it can be straightforwardly extended to a (t, n) -threshold committee deployment with multiple TEEs, improving availability and fault tolerance with only minor modifications. Such multi-TEE architectures have been explored in prior work, e.g., Ekiden [15], and can be combined with generic MPC techniques to distribute secrets across enclaves. Concretely, a multi-TEE deployment can (i) `routerread_data` requests to any available enclave that holds a sufficiently fresh, committee-certified PT snapshot, and (ii) handle policy updates cooperatively within the committee: upon receiving `atable_adminrequest`, the committee enclaves jointly validate the request (including the signature checks and counter/attestation checks in Algorithm 1), execute the update on the PT state, and then co-endorse the resulting PT snapshot by producing attestations from at least t enclaves (or equivalently, a threshold-combined attestation) over the updated $(\gamma_t, \gamma_c, T_{role}, T_{user})$. As a result, the system remains available as long as at least t out of n enclaves are live. Designing an efficient multi-TEE deployment tailored to MISC is an interesting direction for future work.

VII. SECURITY ANALYSIS

We demonstrate how MISC achieves the security goals mentioned in Section III-C.

User Revocation: The primary goal is to ensure that the manager can securely revoke the permissions assigned to a specific data user, even if the cloud server and data users collude. This is achieved by requiring the `table_admin` function to authenticate all administrative requests and verify their timeliness using

the command counter γ_c . In other words, neither a compromised cloud server nor a malicious data user can forge an administrative request or replay an expired one to illegally distribute access to a revoked user. The security of this mechanism depends on both the unforgeability of the signature scheme Ω and the integrity of the TEE. Specifically, if an adversary generates a valid signature without access to the manager's private signing key sk , they could deceive the `table_admin` function into accepting unauthorized administrative commands, implying that the adversary can break the unforgeability of the signature scheme Ω . Similarly, if the adversary could tamper with the code of the `table_admin` function to bypass signature validation, this would represent a breach of the confidentiality and integrity of the TEE.

Confidentiality: Data confidentiality in MISC ensures that sensitive on-chain data is protected from unauthorized access, that authorized users access only what their permissions grant, and that adversaries cannot illegally obtain data or reconstruct sensitive keys. All access to potentially sensitive data is through the TEE, ensuring the computation's confidentiality and integrity. Under a privileged manager, the decryption of on-chain data (using $sk_{2,P}$) and subsequent computation occur within the TEE's isolated environment, protected by TEE confidentiality guarantees. The final result is then re-encrypted using the authorized requesting user's public key (pk_u) via the DT-HPRE scheme Φ before exiting the TEE, ensuring only the intended user can decrypt it; this relies on the semantic security of Φ . Under a constrained manager, original on-chain data is never decrypted to plaintext available to the manager. Re-encryption keys ($rk_{P \rightarrow U}$) are protected (as c_{rk}) by encryption with pk_{att} , decipherable only by the TEE using sk_{att} , relying on the security of the PKE scheme for c_{rk} and TEE confidentiality for sk_{att} . The security of the DT-HPRE scheme Φ ensures that re-encryption (via $\Phi.\text{ReEnc}$) does not reveal the plaintext of re-encrypted on-chain data. In summary, the security of owner secret keys (sk_P) and re-encryption keys (rk) against reconstruction is guaranteed in Φ , while TEE-internal keys ($sk_{att}, sk_{2,P}$) are protected by TEE's confidentiality. The overall confidentiality relies on Φ 's security and the TEE's confidentiality.

Computation Correctness: This property ensures that honest data users receive accurate computation results on secret data and can detect any tampering with parameters or inputs within the TEE. The assurance of accurate computation results stems directly from the TEE's integrity: users retrieve results from authorized operations within this trusted environment, whose integrity is preserved in our system. Turning to the second aspect of this property, the ability to detect tampering with parameters or inputs in TEEs, this capability depends on the soundness of the NIZK scheme Ξ and the unforgeability of the signature scheme Ω . More specifically, if an adversary were able to forge a valid NIZK proof without the decryption key sk , they could deceive the `key_verify` and `table_admin` programs into accepting compromised re-encryption keys or public keys. Such an attack indicates a potential vulnerability in the NIZK scheme, allowing us to construct an adversary against the soundness of the NIZK proof system Ξ . Similarly, if an adversary could generate signatures on policy tables or secret data without access to the hardware key sk_{att} , this would demonstrate a security flaw. In this case, we could construct an adversary against the unforgeability of the signature scheme Ω . Further, we formalize the security of MISC by following the

TABLE I
COMPARISON OF COMPUTATION AND COMMUNICATION COST BETWEEN
DT-HPRE AND ElGAMAL SCHEME [8]

Primitive	DT-HPRE		ElGamal	
	Comp. (s)	Comm. (KB)	Comp. (s)	Comm. (KB)
KeyGen	4.3×10^{-3}	0.77	2.9×10^{-3}	0.13
ReKeyGen	5.9×10^{-4}	0.13	—	—
Enc	4.9×10^{-2}	24.5	3.6×10^{-2}	23.5
ReEnc	4.1×10^{-2}	24.5	—	—
Dec ₁	5.5×10^0	0.25	5.2×10^0	0.25
Dec ₂	3.2×10^{-3}	0.25	—	—
HomAdd	2.2×10^{-4}	24.5	2.2×10^{-4}	23.5

TABLE II
COMPUTATION AND COMMUNICATION COSTS OF PRIMITIVES MEASURED BOTH
OUTSIDE AND INSIDE TEE

Primitive	Notin-TEE		In-TEE (†)	
	Comp. (s)	Comm. (KB)	Comp. (s)	Comm. (KB)
ReEnc	$8.1 \cdot 10^{-2}$	7.0	$8.2 \cdot 10^{-2}$	7.0
Dec ₂	$3.2 \cdot 10^{-3}$	0.01	$3.7 \cdot 10^{-3}$	0.01

†: Measured when executing the primitive inside the TEE.

Global Universal Composability (GUC) framework [38]. We prove that our proposed MISC construction is a GUC secure realization of $\mathcal{F}_{\text{MISC}}$. Here, we will present the formal theorem establishing this security guarantee, while the detailed formal proof is deferred to Appendix A.

Theorem 7.1: Let NIZK Ξ be a non-interactive zero-knowledge scheme, DT-HPRE Ψ be a CPA secure proxy re-encryption scheme, and a PKE be a CPA secure encryption scheme, then the construction in Section VI to UC realizes the ideal functionality $\mathcal{F}_{\text{MISC}}$ in the $(\mathcal{G}_{\text{att}}, \mathcal{G}_{\text{L}})$ -hybrid model.

VIII. EVALUATION

We implemented MISC in C and conducted experiments on a laptop running Ubuntu 20.04 with an Intel Core i5 CPU (8 cores, hyper-threading enabled, 12 execution threads) and 32 GB of RAM. TEEs were realized on the memory-safe, multi-process library OS (LibOS) “Occlum” [42] on Intel SGX. To ensure reliability, each experiment was repeated 20 times, and the average result was used for reporting.

Implementation: We implemented the DT-HPRE scheme based on the construction detailed in Definition 5.1. For cryptographic operations, our implementation leverages a 2048-bit Paillier-based construction and Type A elliptic curves (1024-bit rbits, 1024-bit qbts), employing the PBC library to facilitate pairing computations. The signature scheme Ω was instantiated using Schnorr signatures [43], [44], utilizing the GMP library. Public-key encryption was realized using the ElGamal scheme [20], and ZK-proofs for discrete logarithm tuples were constructed through Σ -protocols, transformed into non-interactive proofs via the Fiat-Shamir heuristic [19]. To evaluate the practicality of MISC, we measured its performance under a constrained manager and a privileged manager.

Benchmark of Primitives: The DT-HPRE scheme is the core cryptographic primitive in MISC, enabling key functionalities such as proxy re-encryption for constrained managers, and direct data processing for privileged managers. To evaluate its practical cost, we implemented DT-HPRE and report the performance of its main operations in Tables I and II. In Table I, we further

TABLE III
COMPUTATION COST (S) FOR INITIALIZATION AND POLICY SETUP OF THE
MISC PROTOCOL

	Initialization		Policy Setup	
	MISC _P	MISC _C	MISC _P	MISC _C
Data user	3.6×10^{-2}	3.6×10^{-2}	—	—
Manager	3.2×10^{-2}	3.2×10^{-2}	3.2×10^{-4}	3.2×10^{-4}
Cloud server	1.1×10^{-2}	1.1×10^{-2}	4.7×10^{-4}	4.7×10^{-4}

compare DT-HPRE with a standard homomorphic encryption baseline (ElGamal) used in prior work [8], with message space set to 2048 bits. The main overhead appears in Enc and Dec₁. Specifically, Enc in DT-HPRE is 49 ms versus 36 ms in the baseline, i.e., approximately 36.1% higher latency, and communication increases from 23.5 KB to 24.5 KB (about 4.2%). For Dec₁, DT-HPRE takes 5.5 s versus 5.2 s, i.e., approximately 5.8% higher latency. For the other operations, differences are minor in practice: KeyGen is within milliseconds, HomAdd is nearly identical in computation time, and DT-HPRE additionally provides ReKeyGen, ReEnc, and two-stage decryption (Dec₁/Dec₂), which are not directly available in the baseline. Although DT-HPRE is moderately slower on key primitives, it enables richer functionality (e.g., proxy re-encryption and double-trapdoor decryption). Importantly, the extra cost is paid only once per invocation of these primitives (e.g., one Enc per ciphertext generation and one Dec₁ per decryption workflow), which keeps the overall system overhead practical. Moreover, Table II shows that executing ReEnc and Dec₂ inside a TEE introduces only negligible additional overhead, supporting the practicality of DT-HPRE in TEE-assisted PSC systems.

MISC Benchmark: We evaluated the performance of MISC by measuring the computation costs across key operational phases — Initialization for system setup, Policy Setup for initializing policy tables, Administration for configuring the tables (i.e., granting a user with a defined role), and Read Data for data users to access on-chain data based on the policy tables—under varying dataset sizes ranging from 100 to 1000 entries. The evaluation covers the MISC_P (with a privileged manager) and MISC_C (with a constrained manager) variants. Particularly, in the Read Data phase, we instantiate $f(v_1, v_2) = v_1 + v_2$ as a representative lightweight computation, where v_1 and v_2 denote the plaintexts of the on-chain ciphertexts. It is worth noting that the execution time in this phase is dominated by cryptographic operations (e.g., DT-HPRE decryption) rather than plaintext computations within the TEE. Consequently, substituting f with more complex functions incurs negligible additional overhead. Additionally, we evaluated the latency of Read Data operations with 5 to 100 concurrent users, where multiple users simultaneously issued Read Data requests on a server with an INTEL(R) XEON(R) GOLD 5520+ and 256 GB of RAM.

Initialization and Policy Setup: Table III presents the computation costs for Initialization and Policy Setup under both MISC_P and MISC_C constructions. For Initialization, each data user and managers respectively, exhibit 36 ms and 32 ms. Cloud servers achieve faster initialization, requiring 11 ms and 11 ms, respectively. The Policy Setup phase introduces negligible overhead, with both constructions completing in 0.32 ms (manager) and 0.47 ms (cloud server). These results indicate that the one-time costs incurred by each party during Initialization and Policy Setup are modest and dataset-independent, hence do not bottleneck practical performance. *Operational Phases.*

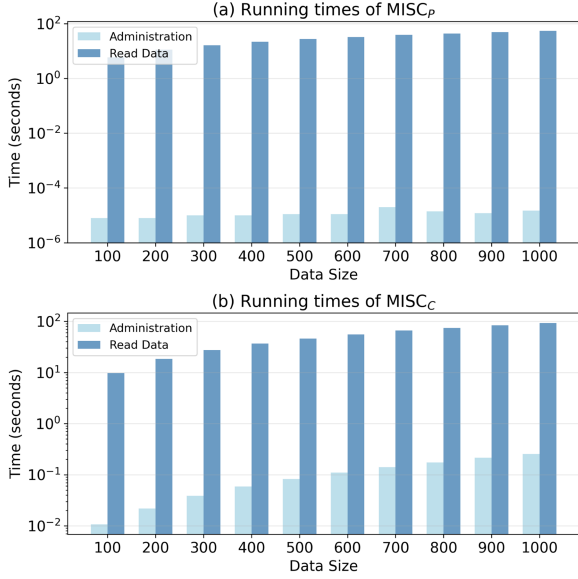


Fig. 3. Running times of MISC: The x-axis represents the dataset size, while the y-axis indicates the execution time on a logarithmic scale.

Administration and Read Data: Fig. 3 illustrates the running times for the Administration and Read Data phases. The results demonstrate predictable scalability. The computation time for the Read Data phase grows linearly with the dataset size for both $MISC_P$ (Fig. 3(a)) and $MISC_C$ (Fig. 3(b)), increasing from approximately 5.9 s and 9.7 s for 100 entries to 54.8 s and 92.7 s for 1000 entries. This linear relationship indicates that MISC can handle larger datasets with predictable performance degradation. The Administration phase also exhibits linear growth, but with significantly lower absolute times compared to Read Data. For $MISC_P$ (Fig. 3(a)), administration time increases from 6 μ s to 15 μ s. For $MISC_C$ (Fig. 3(b)), the time is higher, ranging from 9.5 ms to 255 ms, reflecting the additional overhead within the TEE for managing proxy re-encryption keys and associated policies specific to the constrained model. However, even in $MISC_C$, these administration times remain practical for managing user access and permissions.

Read Data Latency under Concurrent User Access: Fig. 4 shows the read latency distribution as the number of concurrent users increases from 5 to 100, where each user issues a read request over 100 data points. For $MISC_P$ in Fig. 4(a), the average latency increases from 9.37 s at 5 users to 23.70 s at 100 users, exhibiting smooth and gradual growth. Moreover, the average, P50, and P95 curves remain tightly clustered (e.g., at 100 users: avg=23.70 s, P50=24.04 s, P95=25.42 s), indicating that increasing concurrency does not introduce substantial latency dispersion. For $MISC_C$ in Fig. 4(b), the read latency grows from 9.86 s at 5 users to 40.01 s at 100 users. Despite this steeper increase, the average, P50, and P95 values remain close across all concurrency levels (e.g., at 100 users: avg=40.01 s, P50=40.94 s, P95=42.81 s), suggesting a stable latency distribution and limited tail amplification under load. These results indicate that the system maintains predictable read performance, even when serving a large number of concurrent users.

Implications for Practicality and Deployment: The benchmark results confirm the practical efficiency and scalability

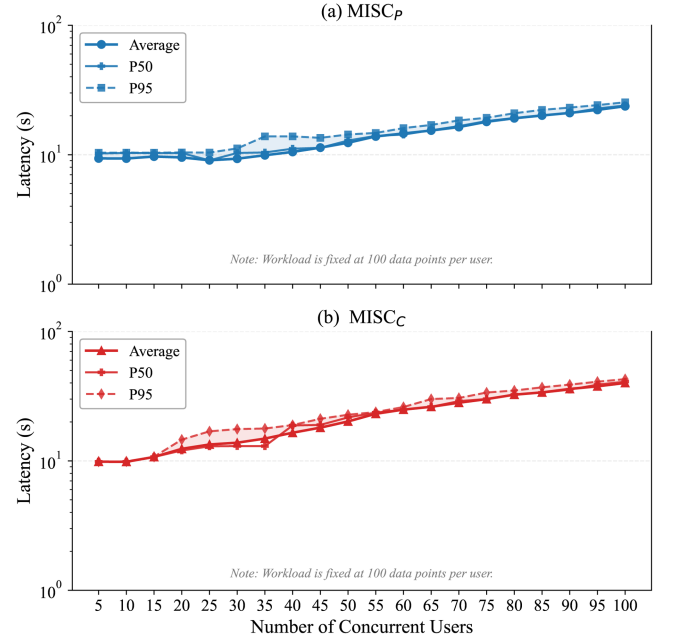


Fig. 4. Latency Distribution of Read Data operations under varying concurrency levels (1 to 100 users). The solid lines represent the average latency, while the other curves correspond to P50 (i.e., 50% of requests complete no slower than this value) and P95 latencies.

of MISC. The low setup costs (Initialization, Policy Setup) facilitate easy adoption. The linear scaling of operational phases (Administration, Read Data) ensures predictable performance as data volume grows. While Read Data operations involving computation over entire datasets take tens of seconds for several hundred entries, these times are well within acceptable limits for many real-world applications involving batch processing or periodic analysis, where immediate real-time responses per record are not the primary requirement. The efficiency of the Administration phase further supports smooth operational workflows. Overall, the performance characteristics demonstrate that MISC is a practical solution for enhancing PSCs with flexible, TEE-assisted data management capabilities adaptable to different trust requirements.

IX. RELATED WORKS

Table IV summarizes the assumptions and properties of the state-of-the-art in managing on-chain data and makes a comparison with our work.

Managing on-chain data while ensuring flexible access mode, user revocation, as well as minimal on-chain cost, and supporting various privilege levels for managers remains a significant challenge, particularly in off-chain environments. Private smart contract schemes can also be applied to manage on-chain data. Some schemes like Hawk [9], Ekiden [15], and Jigsaw [45] support user revocation but are not compatible with Ethereum. Although RaceTEE [46], [47], [48] achieves Ethereum compatibility and supports flexible revocation through key rotation, it still requires on-chain transactions for every invocation, thereby failing to eliminate Gas costs for users. Similarly, TrustFabric [49] introduces a TEE-based privacy-preserving method for Hyperledger Fabric by offloading execution to an off-chain TEE cluster.

TABLE IV

STATE-OF-THE-ART IN MANAGING ON-CHAIN DATA. WE INDICATE WHICH TOOLS SUPPORT FLEXIBLE ACCESS MODE (⚙️), PROVIDE USER REVOCATION (🔑), RELY ON MINIMAL ON-CHAIN COST (\$), AND SUPPORT MANAGERS WITH VARYING PRIVILEGE LEVELS (👤).

	Tool	⚙️	🔑	\$	👤
On-chain	Hawk [9]	●	○	○	○
	Ekiden [15]	●	○	○	○
	Zkay [7]	○	○	○	○
	Zeestar [8]	○	○	○	○
	RaceTEE [46]–[48]	●	●	○	○
	TrustFabric [49]	●	●	○	○
	Jigsaw [45]	●	○	○	○
Off-chain	CALYPSO [11]	○	●	○	○
	Oasis ¹ [13]	●	●	●	○
	Ours	●	●	●	●

¹ An additional centralized storage space is required.

While it supports revocation via key rotation and flexible access via mission contracts, it still relies on on-chain interactions for mission submission and result verification, which does not alleviate the on-chain overhead. Other approaches, like Zkay [7] and Zeestar [8], restrict data access solely to the owner, making them suitable only for limited use cases. A common limitation across these solutions is that each permission assignment generates an on-chain transaction, thereby not providing scalability guarantees.

For the off-chain setting, while some efforts [8], [11], [13] have been made in this area, comprehensive exploration is still lacking. In particular, CALYPSO [11] relies on semi-honest committees for key management, which exposes data privacy to Byzantine adversaries [50]. Moreover, CALYPSO requires the decryption of encrypted data before performing calculations, thereby compromising the on-chain data privacy. Further, in the industrial context, the Oasis Parcel [13] solution implements data access control by building a centralized SQL database, which demands system assumptions such as the existence of additional storage space and a stable data synchronization mechanism. Thus, rebuilding a centralized database is unsuitable for extending data management in current PSCs.

X. CONCLUSION AND FUTURE WORK

This paper presents a comprehensive and practical solution for secure on-chain data management by designing the Double-Trapdoor Homomorphic Proxy Re-encryption (DT-HPRE) scheme, integrated within a TEE-assisted framework called MISC. Our solution addresses critical challenges such as achieving flexible permission mode with minimal overhead and supporting managers with varying trust levels when managing on-chain data in PSCs, which are not considered simultaneously in previous works. The formal security analysis conducted under the GUC framework rigorously validates the scheme's security. Moreover, our implementation demonstrates that the proposed solution is computationally efficient and highly scalable, making it suitable for real-world deployment. In future work, we aim to enhance security, performance, and functionality. We will explore more efficient fully homomorphic proxy re-encryption schemes to enable more flexible computations on ciphertexts and support more dynamic access policies. Furthermore, we plan to investigate the security of our framework under weaker

TEE models, i.e., where the confidentiality of service/attestation keys is preserved, but parts of the TEE's internal state may be observable.

ACKNOWLEDGMENT

Any opinions, findings and conclusions or recommendations expressed in this material are those of the author(s) and do not reflect the views of National Research Foundation, Singapore and Infocomm Media Development Authority."

REFERENCES

- [1] B. Parno, J. Howell, C. Gentry, and M. Raykova, "Pinocchio: Nearly practical verifiable computation," in *Proc. IEEE Symp. Secur. Privacy*, 2013, pp. 238–252.
- [2] V. Costan and S. Devadas, "Intel SGX explained," *Cryptol. ePrint Arch.*, 2016.
- [3] A. H. Seh et al., "Healthcare data breaches: Insights and implications," *Healthcare*, vol. 8, no. 22020, Art. no. 133.
- [4] R. Li, Q. Wang, Q. Wang, D. Galindo, and M. Ryan, "SoK: TEE-assisted confidential smart contract," in *Proc. Privacy Enhancing Technol.*, 2022, vol. 3, pp. 711–731.
- [5] M. Li et al., "IvyApc: Auditable generalized payment channels," in *Proc. Int. Conf. Financial Cryptogr. Data Secur.*, 2025, pp. 39–55.
- [6] Y. Li et al., "AuditPCH: Auditable payment channel hub with privacy protection," *IEEE Trans. Inf. Forensics Secur.*, vol. 20, pp. 1251–1261, 2025.
- [7] S. Steffen, B. Bichsel, M. Gersbach, N. Melchior, P. Tsankov, and M. Vechev, "zkay: Specifying and enforcing data privacy in smart contracts," in *Proc. ACM SIGSAC Conf. Comput. Commun. Secur.*, 2019, pp. 1759–1776.
- [8] S. Steffen, B. Bichsel, R. Baumgartner, and M. Vechev, "ZeeStar: Private smart contracts by homomorphic encryption and zero-knowledge proofs," in *Proc. IEEE Symp. Secur. Privacy*, 2022, pp. 179–197.
- [9] A. Kosba, A. Miller, E. Shi, Z. Wen, and C. Papamanthou, "Hawk: The blockchain model of cryptography and privacy-preserving smart contracts," in *Proc. IEEE Symp. Secur. Privacy*, 2016, pp. 839–858.
- [10] J. Ernstberger et al., "SoK: Data sovereignty," *Cryptol. ePrint Arch.*, 2023.
- [11] E. K. Kogias, E. C. Alp, L. Gasser, P. S. Jovanovic, E. Syta, and B. A. Ford, "CALYPSO: Private data management for decentralized ledgers," *Proc. VLDB Endowment*, vol. 14, pp. 586–599, 2021.
- [12] G. Almashaqbeh and R. Solomon, "SoK: Privacy-preserving computing in the blockchain era," in *Proc. Eur. Symp. Secur. Privacy*, 2022, pp. 124–139.
- [13] Oasis Labs, "Oasis labs - Privacy-first, high performance cloud computing on blockchain," 2023. Accessed: Feb. 22, 2023. [Online]. Available: <https://www.oasislabs.com/>
- [14] "ARM trustzone for Cortex-M," Accessed: Mar. 14, 2023. [Online]. Available: <https://www.arm.com/technologies/trustzone-for-cortex-m>
- [15] R. Cheng et al., "Ekiden: A platform for confidentiality-preserving, trustworthy, and performant smart contracts," in *Proc. Eur. Symp. Secur. Privacy*, 2019, pp. 185–200.
- [16] E. Bresson, D. Catalano, and D. Pointcheval, "A simple public-key cryptosystem with a double trapdoor decryption mechanism and its applications," in *Proc. Int. Conf. Theory Appl. Cryptol. Inf. Secur.*, 2003, pp. 37–54.
- [17] J. Katz, U. Maurer, B. Tackmann, and V. Zikas, "Universally composable synchronous computation," in *Proc. Theory Cryptogr. Conf.*, 2013, pp. 477–498.
- [18] A.-A. A. Sharabati and E.R. Jreisat, "Blockchain technology implementation in supply chain management: A literature review," *Sustainability*, vol. 16, 2024, Art. no. 2823.
- [19] P. Chaidos and J. Groth, "Making Sigma-protocols non-interactive without random oracles," in *Proc. IACR Int. Workshop Public Key Cryptogr.*, 2015, pp. 650–670.
- [20] Y. Tsounis and M. Yung, "On the security of ElGamal based encryption," in *Proc. Int. Workshop Public Key Cryptogr.*, 1998, pp. 117–134.
- [21] R. Kaur and A. Kaur, "Digital signature," in *Proc. Int. Conf. Comput. Sci.*, 2012, pp. 295–301.
- [22] R. Pass, E. Shi, and F. Tramer, "Formal abstractions for attested execution secure processors," in *Proc. Annu. Int. Conf. Theory Appl. Cryptographic Techn.*, 2017, pp. 260–289.

- [23] C. Badertscher, U. Maurer, D. Tschudi, and V. Zikas, "Bitcoin as a transaction ledger: A composable treatment," in *Proc. Annu. Int. Cryptol. Conf.*, 2017, pp. 324–356.
- [24] V. Buterin, "Ethereum: A next-generation smart contract and decentralized application platform," Ethereum White Paper, 2014. Accessed: Jun. 16, 2026. [Online]. Available: <https://ethereum.org/whitepaper/>
- [25] M. Graf, D. Rausch, V. Ronge, C. Egger, R. Küsters, and D. Schröder, "A security framework for distributed ledgers," in *Proc. ACM SIGSAC Conf. Comput. Commun. Secur.*, 2021, pp. 1043–1064.
- [26] J.-K. Zinzindohoué, K. Bhargavan, J. Protzenko, and B. Beurdouche, "HACL*: A verified modern cryptographic library," in *Proc. ACM SIGSAC Conf. Comput. Commun. Secur.*, 2017, pp. 1789–1806.
- [27] I. Bentov, Y. Ji, F. Zhang, L. Breidenbach, P. Daian, and A. Juels, "Tesseract: Real-time cryptocurrency exchange using trusted hardware," in *Proc. ACM SIGSAC Conf. Comput. Commun. Secur.*, 2019, pp. 1521–1538.
- [28] Y. Xu, W. Cui, and M. Peinado, "Controlled-channel attacks: Deterministic side channels for untrusted operating systems," in *Proc. IEEE Symp. Secur. Privacy*, 2015, pp. 640–656.
- [29] J. V. Bulck et al., "Foreshadow: Extracting the keys to the Intel SGX kingdom with transient out-of-order execution," in *Proc. USENIX Secur.*, 2018, pp. 991–1008.
- [30] M.-W. Shih, S. Lee, T. Kim, and M. Peinado, "T-SGX: Eradicating controlled-channel attacks against enclave programs," in *Proc. Annu. Netw. Distrib. Syst. Secur. Symp.*, 2017, pp. 15–43.
- [31] V. Costan, I. Lebedev, and S. Devadas, "Sanctum: Minimal hardware extensions for strong software isolation," in *Proc. USENIX Secur.*, 2016, pp. 857–874.
- [32] S. Chen, X. Zhang, M. K. Reiter, and Y. Zhang, "Detecting privileged side-channel attacks in shielded execution with déjà vu," in *Proc. ACM Asia Conf. Comput. Commun. Secur.*, 2017, pp. 7–18.
- [33] S. Van Schaik, A. Kwong, D. Genkin, and Y. Yarom, "SGXax: How SGX fails in practice," 2020. [Online]. Available: <https://sgaxe.com/files/SGXax.pdf>
- [34] R. Walther, C. Weinhold, and M. Roitzsch, "RATLS: Integrating transport layer security with remote attestation," in *Proc. Int. Conf. Appl. Cryptogr. Netw. Secur.*, 2022, pp. 361–379.
- [35] K. Murdock, D. Oswald, F. D. Garcia, J. Van Bulck, D. Gruss, and F. Piessens, "Plundervolt: Software-based fault injection attacks against Intel SGX," in *Proc. IEEE Symp. Secur. Privacy*, 2020, pp. 1466–1482.
- [36] A. Nilsson, P. N. Bideh, and J. Brorsson, "A survey of published attacks on Intel SGX," 2020, *arXiv:2006.13598*.
- [37] B. Liu and B. Warinschi, "Universally composable cryptographic role-based access control," in *Proc. Int. Conf. Provable Secur.*, 2016, pp. 61–80.
- [38] R. Canetti, Y. Dodis, R. Pass, and S. Walfish, "Universally composable security with global setup," in *Proc. 4th Theory Cryptography Conf. (TCC 2007)*, Amsterdam, The Netherlands, Feb. 21–24, 2007, in Lecture Notes in Computer Science, S. P. Vadhan, Ed., Berlin, Heidelberg: Springer, 2007, pp. 61–85, doi: [10.1007/978-3-540-70936-7_4](https://doi.org/10.1007/978-3-540-70936-7_4).
- [39] R. Canetti, "Universally composable signature, certification, and authentication," in *Proc. IEEE Comput. Secur. Foundations Workshop*, 2004, pp. 219–233.
- [40] R. Canetti and S. Hohenberger, "Chosen-ciphertext secure proxy re-encryption," in *Proc. ACM Conf. Comput. Commun. Secur.*, 2007, pp. 185–194.
- [41] H. Shafagh, A. Hithnawi, L. Burkhalter, P. Fischli, and S. Duquennoy, "Secure sharing of partially homomorphic encrypted IoT data," in *Proc. ACM Conf. Embedded Netw. Sensor Syst.*, 2017, pp. 1–14.
- [42] Y. Shen et al., "Occlum: Secure and efficient multitasking inside a single enclave of Intel SGX," in *Proc. Int. Conf. Architectural Support Program. Languages Operating Syst.*, 2020, pp. 955–970.
- [43] C.-P. Schnorr, "Efficient signature generation by smart cards," *J. Cryptol.*, vol. 4, pp. 161–174, 1991.
- [44] A. Menezes and A. Menezes, "The discrete logarithm problem," in *Elliptic Curve Public Key Cryptosystems*. Boston, MA, USA: Springer, 1993, pp. 49–59.
- [45] S. Garg, A. Goel, D. Kolonelos, and R. Sinha, "Jigsaw: Doubly private smart contracts," *Cryptol. ePrint Arch.*, 2025.
- [46] K. Zhang and A. Martin, "RaceTEE: Enabling interoperability of confidential smart contracts," in *Proc. TrustCom*, 2025, pp. 62–73.
- [47] J. Gebele, T. Mutzel, B. Oez, and F. Matthes, "Cross-chain sealed-bid auctions using confidential compute blockchains," 2025, *arXiv:2510.19491*.
- [48] J. Gebele, T. Mutzel, B. Öz, and F. Matthes, "Sealed-bid auctions via TEE-backed confidential compute blockchains," in *Proc. Workshop Decentralized Finance Secur.*, 2025, pp. 16–26.
- [49] D. Cao, B. Li, J. He, L. Wang, L. Liao, and Y. Zhou, "TrustFabric: A privacy-preserving method for hyperledger fabric using trusted execution environment," in *Proc. IEEE 31th Int. Conf. Parallel Distrib. Syst.*, 2025, pp. 1–10.
- [50] S. Jaggi, M. Langberg, S. Katti, T. Ho, D. Katabi, and M. Médard, "Resilient network coding in the presence of byzantine adversaries," in *Proc. IEEE Int. Conf. Comput. Commun.*, 2007, pp. 616–624.



cryptography.



& Secure Computing. His research interests include trusted computing, secure computation, and public-key cryptography.



Guomin Yang (Senior Member, IEEE) received the PhD degree in computer science from the City University of Hong Kong in 2009. He was an associate professor with the University of Wollongong, Australia. He is currently an associate professor with the School of Computing and Information Systems, Singapore Management University. His research interests include applied cryptography and privacy enhancing technologies. He is an associate editor for *IEEE Transactions on Information Forensics and Security* and *IEEE Transactions on Services Computing*.



Haoxin Tu (Member, IEEE) is currently a research fellow with the National University of Singapore, working with prof. Abhik in the Trustworthy and Secure Software group. His work has been authored or coauthored at top venues including IEEE S&P, FSE, and ACM TOSEM. His research interests include improving the reliability and security of software systems through fuzzing, symbolic execution, and program analysis. His recent work extends to agentic software verification, where intelligent agents collaborate with theorem provers to construct machine-checked proofs.



Jianfei Sun received the PhD degree from the University of Electronic Science and Technology of China. He was a research fellow with the School of Computer Science and Engineering, Nanyang Technological University. He is currently a research scientist with Singapore Management University. He has authored or coauthored many papers on *IEEE Transactions on Dependable and Secure Computing*, *IEEE Transactions on Information Forensics and Security*, *IEEE Transactions on Industrial Informatics*, *IEEE Transactions on Cloud Computing*, and *IEEE Transactions on Vehicular Technology*. His research interests include network security and IoT security. He is an associate editor for *IEEE Transactions on Industrial Informatics*.



Robert H. Deng (Fellow, IEEE) has been an AXA chair professor of cybersecurity and a professor of information systems with the School of Information Systems, Singapore Management University, since 2004. His research interests include data security and privacy, multimedia security, and network and system security. He was the recipient of the Distinguished Paper Award (NDSS 2012), Best Paper Award (CMS 2012), and Best Journal Paper Award (IEEE Communications Society 2017). He is/was on the editorial boards of many international journals including *IEEE Transactions on Information Forensics and Security*, *IEEE Transactions on Dependable and Secure Computing*, and *IEEE Transactions on Knowledge and Data Engineering*.